

University of Wollongong
CSIT305: Emerging Information Technologies and their Applications

ASSESSMENT COVER SHEET

Student Name	Student Number	Tutorial
Joseph Koh (Leader)	1441309	T01
Suzaina	1037195	T01
Khin Kan Kaw Lin	1485064	T01
May Yu Ya Maw	8950362	T01
Vincent	1150005	T01

Tutor name: Mr. Liaw
Assessment: Group Project

STUDENTS DECLARATION

I declare that this assessment is entirely my own work and has not been submitted for assessment in any other subject or course. This work complies with the University of Wollongong's Academic Integrity Policy.

Signature: *Joseph* *suzaina* *may*  

Date: 24 Aug 2026

Team Contribution

Student Name	Workload	Contribution (%)
Joseph (Leader)	System Overview - Introduction, System Goal, Stakeholders, Existing solution and Comparison Function 1 (AI-Based Flood Prediction & Risk Analysis)	22%
Khin Kan Kaw Lin	Set Up Presentation Slides Function 2 (Drainage and Pump Health Monitoring)	22%
May	Project Report Compilation FloodWatch Architecture Function 3 (Early Warning & Multi-Channel Alert Dissemination)	22%
Suzaina	Project Report Compilation Function 4 (Stormwater Storage Optimisation)	22%
Vincent	Function 5 (Tactical Ground Situational Awareness & Operational Feeds)	12%

Table Of Contents

1. Introduction	4
2. System Goal	4
4. Existing Solutions & Comparison	4
5. FloodWatch Architecture	7
6. Key Functions	8
6.1 AI-Based Flood Prediction & Risk Analysis	9
6.1.1 Multi Modal Database	9
6.1.2 Dataset Query Processing	13
6.1.3 Software Engineering	15
6.1.4 Networking	20
6.1.5 Advanced Technologies and Security	22
6.2 Drainage & Pump Health Monitoring	24
6.2.1 Significance	24
6.2.2 Multi-Modal Database	25
6.2.3 Networking Infrastructure	27
6.2.4 Software Architecture	28
6.3 Early Warning & Multi-Channel Alert Dissemination	31
6.3.1 Functional Overview	31
6.3.2 Multi-Modal Database	31
6.3.3 Software Engineering	35
6.3.4 Networking	37
6.4 Stormwater Storage Optimisation	40
6.4.1 Operational Workflow	40
6.4.2 Multimodal Database	40
6.4.3 Software Design & Architecture	43
6.4.4 Network Infrastructure and Integration	46
6.5 Tactical Ground Situational Awareness & Operational Feeds	49
6.5.1 Multi-Modal Database	49
6.5.2 Software Design & Architecture	51
6.5.3 Networking Infrastructure & Integration	52
6.5.4 Advanced Technologies & Security	53
7. References	54

1. Introduction

Singapore has a tropical climate and receives frequent rainfall of approximately 2000 - 2500mm annually. As a highly urbanised city, much of its land is covered by impermeable surfaces such as roads, buildings, and concrete pavements. These surfaces prevent rainfall from infiltrating into the ground, increasing surface runoff during periods of heavy rainfall. Large volumes of runoff can overwhelm drainage systems and waterways resulting in flash floods in densely developed areas (CBD).

A recent study found that nearly all 108 urban flooding events recorded in Singapore between 2017 and 2023 were associated with short duration, high intensity rainfall, which overwhelmed drainage systems and resulted in flash floods (Lommen et al., 2025).

Therefore, **FloodWatch** is proposed as a smart flood management system by team Mag4 to mitigate the ever increasing risk of flash flooding in Singapore. The system leverages on emerging information technologies and its applications to improve and enhance the current flood management system in Singapore.

2. System Goal

The primary goal of FloodWatch is to rearchitect the current existing flash flood management system infrastructure in Singapore by transforming it from a reactive approach to a proactive and preventive approach. The system harnesses emerging cutting-edge technologies such as IoT devices, advanced databases, next generation networking infrastructure, predictive AI machine learning models and modern cloud platforms to enable real-time monitoring, analysis, and decision making. By providing accurate and timely information, FloodWatch enables stakeholders and the public to make quick informed decisions, strengthen emergency preparedness and improve public safety.

3. Stakeholders

Users / Agencies	Role	Use case
Public Utilities Board (PUB)	System User	Primary user of FloodWatch to monitor real-time data and predictive analysis
National Environmental Agency (NEA)	Data Provider	Primary data provider who provides real-time and historical weather and rainfall data
Meteorological Service Singapore (MSS) (under NEA)	System Operator	Operates weather radar systems to produce rainfall forecasts, heavy rain warnings, and storm predictions
Singapore Civil Defence Force(SCDF)	Rescue Operator	Conducts evacuations and rescue operations on severe flooding events
Land Transport Authority (LTA)	Infrastructure Manager	Manages traffic closure and diversion in flood affected areas
Certis CISCO under Singapore Police Force (SPF)	Traffic Police	Assists in traffic control and public safety in flood affected areas
Town Councils	Estate Manager	Maintain estate drainage systems in residential areas

Singapore's flood management system is managed by multiple government agencies with the Public Utilities Board (PUB) as the lead agency. Figure 1 illustrates a high level overview of Singapore's current flood management system and the coordination between key stakeholders.

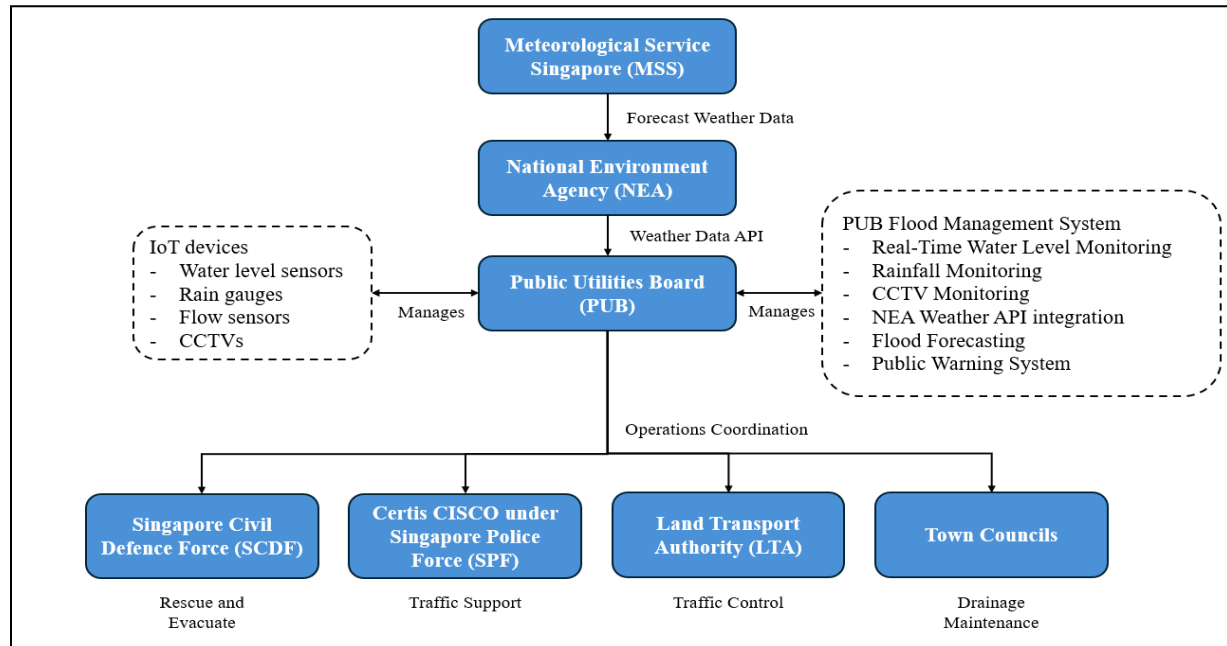


Figure 1. High-level overview of Singapore's existing flood management system (author's illustration, adapted from PUB, 2025).

PUB's current flood management system relies on real-time monitoring, rainfall forecasting and public alerts to detect potential flood locations and support timely response. Its Joint Operations Centre (JOC) monitors sensor and weather data around the clock and deploys Quick Response Teams to affected areas (PUB 2026).

Figure 2 illustrates a high level process overview of how PUB manages the flood management system.

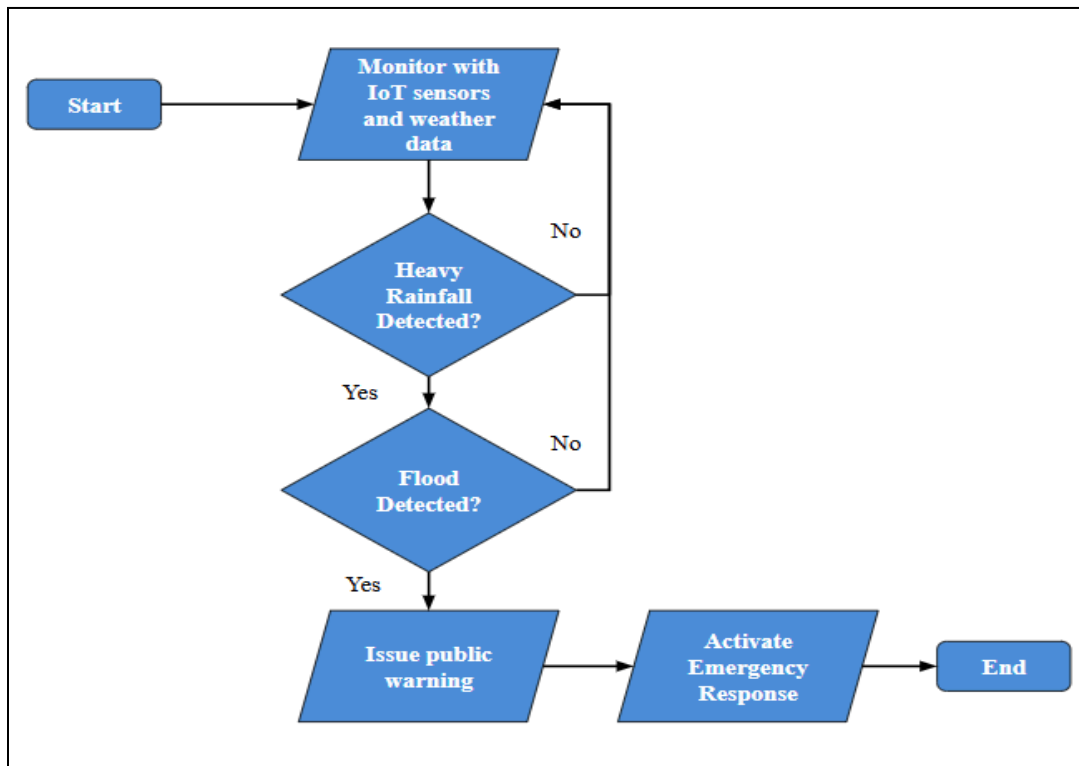


Figure 2. High-level flow diagram of Singapore's existing flood management system (author's illustration, adapted from PUB, 2025).

Although the current system provides real-time monitoring, it relies on monitoring existing conditions rather than predicting flash flood risks before they occur. This reactive approach motivates the development of FloodWatch, which integrates multiple functions to enable proactive flood management.

Figure 3 illustrates a high level process overview of FloodWatch .

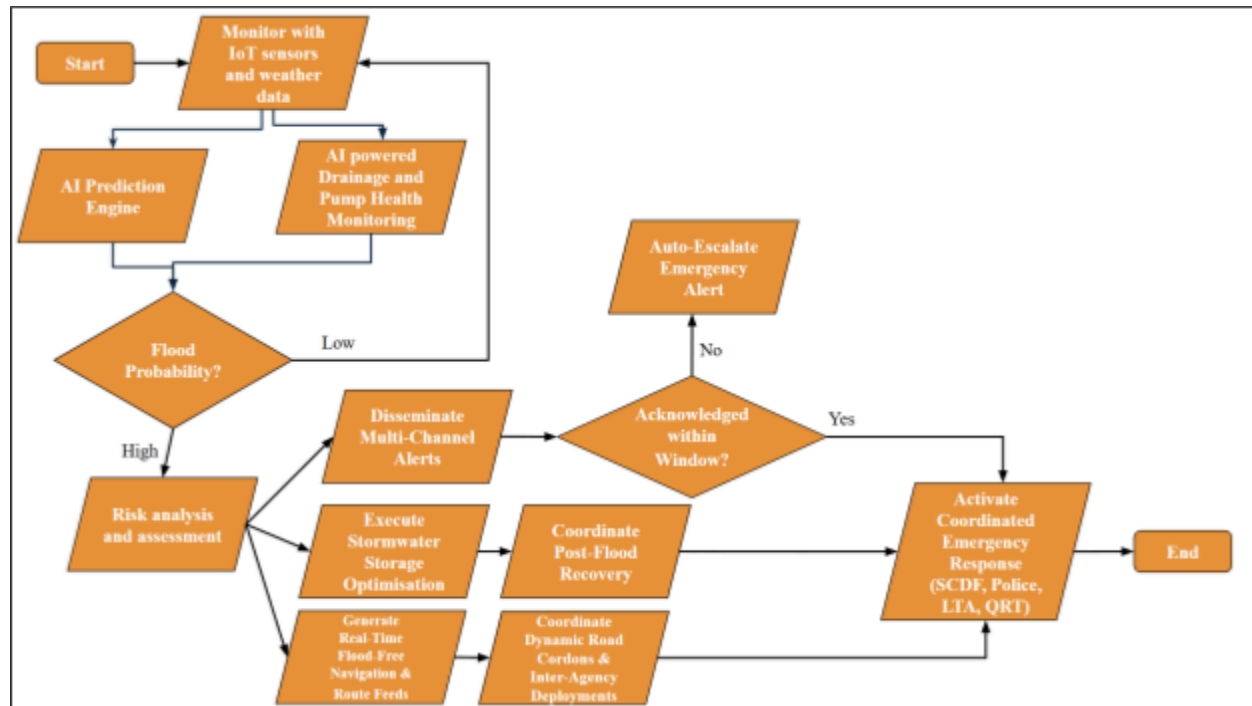


Figure 3. High-level flow diagram of FloodWatch

FloodWatch restructures the existing approach by adopting an Anticipatory Preventive Model that leverages predictive analytics, risk assessment, and many more to identify potential flooding in advance, enabling proactive interventions, informed decision making, and preventive measures that enhances overall flood resilience.

5. FloodWatch Architecture

The figure below illustrates the overall system architecture underpinning FloodWatch. This diagram shows the single, shared AWS-based platform that all five key functions are built on top of: a common data-ingestion layer, a shared multi-modal database tier, and a common dissemination layer. This shared-platform design is what allows the five functions to interoperate as one integrated system rather than as five independent tools, for example, the Early Warning function (Section 6.3) reads directly from the same Timestream and DynamoDB tables the Flood Prediction function (Section 6.1) writes to, without needing a separate integration layer.

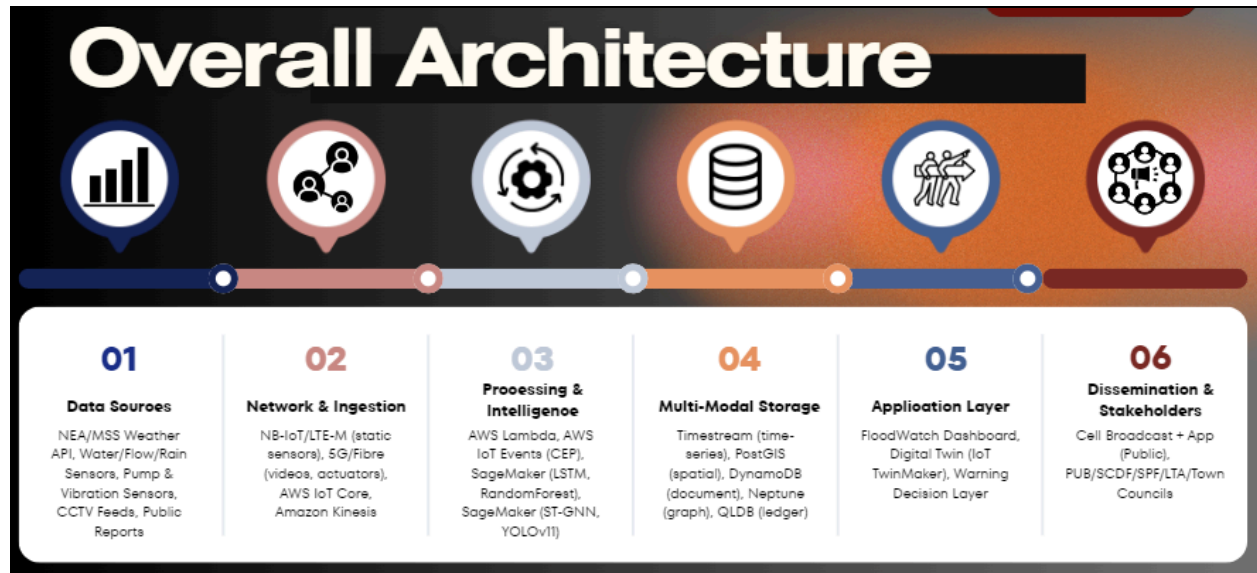


Figure 4. FloodWatch overall system architecture, showing the shared data, network, processing, storage, application and dissemination layers used across all five key functions (author's illustration).

6. Key Functions

The key functions of FloodWatch are as follows:

1. AI-Based Flood Prediction & Risk Analysis
2. AI-Powered Drainage & Pump Health Monitoring
3. Early Warning & Multi-Channel Alert Dissemination
4. Stormwater Storage Optimisation
5. Tactical Ground Situational Awareness & Operational Feeds Monitoring System

6.1 AI-Based Flood Prediction & Risk Analysis

The purpose of this function is to predict flood events and perform impact analysis before flooding occurs. Unlike conventional flood management systems which alert stakeholders after flooding has already begun, this function alerts stakeholders of flood probabilities well in advance giving response teams sufficient time to implement preventive measures, coordinate emergency operations and also execute damage control increasing overall efficiency of flood management.

Real-time environmental data is continuously collected from IoT devices deployed across Singapore such as water level sensors, rain gauges, flow sensors, CCTV cameras and weather forecast APIs. The collected data is transmitted through an IoT gateway to a cloud platform where it is stored in multiple noSQL databases. Using real-time streaming processing, the data is analysed by AI based supervised machine learning models that combine live sensor readings, weather forecasts and historical records to predict flood probability and estimate flood severity.

Additionally, this function also performs detailed flood impact analysis on the predicted high risk areas. This is achieved by representing the predicted flood as a geospatial flood map which is overlaid with Geographic Information System (GIS) layers containing information on MRT stations, roads, buildings and drainage networks. By performing spatial analysis, the system then identifies assets that are likely to be affected, estimates disruption levels and generates actionable suggestions such as alternative travel routes, temporary road closure or emergency response team deployment.

This enables stakeholders to make informed, data-driven decisions before flooding occurs, reducing risks to infrastructure and the public.

6.1.1 Multi Modal Database

Dataset Storage

S/ N	Data	Update Frequency	Data Source	Data Destination	Database Type
1	2-Hour Weather Forecast	Every 30 mins	NEA REST API	AWS RDS Postgres SQL	SQL
2	2-Hour Weather Forecast	Every 30 mins	NEA REST API	AWS ElastiCache (Redis)	NoSQL (Cache)
3	Water Level Sensor, Flow Sensor, Rain Sensor	Real-Time	IoT Devices (Sensors)	AWS Timestream	Time-Series DB

4	Sensor Location	Static (On Registration)	IoT Devices (Sensors)	AWS RDS PostgreSQL + PostGIS	Spatial DB
5	GIS Data (Buildings, Roads, Rivers, Flood Zones)	As Required	GIS Dataset	AWS RDS PostgreSQL + PostGIS	Spatial DB
6	Flood Prediction Report and Incident Report	Event-driven	AI Prediction Engine	AWS DynamoDB	Document DB

There are 4 types of data:

- NEA REST API Data
- Live IoT Sensor Readings and Location Data
- GIS Data
- AI Prediction Engine Data

NEA REST API Data

2 datasets containing 2-hour weather forecast data will be retrieved from NEA REST API, with each dataset serving a different purpose.

- AWS ElastiCache (Redis): Each data retrieval replaces the previous record, keeping only the latest snapshot of the weather forecast. This enables fast access for the AI prediction engine which answers “what is the current weather?”
- AWS Postgres SQL: Each data retrieval inserts a new record keeping a historical archive of weather forecast snapshots. This supports historical data analysis, trend detection, model training, reporting and also auditing.

Figure below illustrates an example of data stored in AWS ElastiCache (Redis) and AWS Postgres SQL:

1. AWS ElastiCache (Redis):

Key: weather:Tampines
Value:
{
"forecast": "Light Rain",
"temperature": 29,
"humidity": 78,
"updated_at": "2026-07-26T10:30:00Z"
}

2. AWS Postgres SQL:

id	area	forecast	temperature	humidity	forecast_timestamp
1	Tampines	Cloudy	30	75	7/26/2026 8:00
2	Tampines	Light Rain	29	78	7/26/2026 8:30
3	Tampines	Moderate Rain	28	82	7/26/2026 9:00

Live IoT Sensor Readings and Location Data

Each sensor periodically transmits readings at 1-minute intervals which are ingested into AWS Timestream (Time-Series DB). Each reading is stored as a timestamped record, providing a real-time time-series dataset for the AI prediction engine.

Figure below illustrates an example of real-time sensor readings stored in AWS Timestream:

Time	Sensor ID	Sensor Type	Location	Measurement	Value	Unit
7/26/2026 10:00	WL001	WaterLevel	Bedok Reservoir	water_level	2.35	m
7/26/2026 10:00	RN001	Rain	Yishun MRT Station	rainfall	4.2	mm
7/26/2026 10:00	FL001	Flow	Drain A	flow_rate	12.5	m ³ /s
7/26/2026 10:01	WL001	WaterLevel	Canal C	water_level	2.38	m
7/26/2026 10:01	RN001	Rain	Orchard MRT Station	rainfall	4.6	mm

IoT sensors' location data are stored in AWS RDS PostgreSQL with PostGIS extension enabled. PostGIS supports storing geometry objects in addition to normal relational data. A geometry object in PostGIS is a set of coordinates that define the shape and location of a geographic feature:

- POINT()
 - IoT sensor devices' location are stored as POINT geometries
- Polygon()
 - Buildings and flood zones are stored as Polygon geometries
- LineString()
 - Roads and rivers are stored as LineString geometries

sensor_id	sensor_type	latitude	longitude	location (Geometry)	installation_date	status
WL001	Water Level	1.3521	103.8198	POINT(103.8198 1.3521)	7/26/2026	Active
RN001	Rain	1.3545	103.821	POINT(103.8210 1.3545)	7/26/2026	Active
FL001	Flow	1.3502	103.818	POINT(103.8180 1.3502)	7/26/2026	InActive

Figure 5. An example of IoT sensors' location stored in AWS RDS PostgreSQL+ PostGIS extension

GIS Data

GIS datasets are stored as separate spatial tables depending on their geometry type. This dataset is crucial as it serves as the Source of Truth (SOT) for GIS overlay analysis. However, the GIS database must first be populated with spatial data before it can perform spatial queries. The

master data is provided by Government GIS datasets, OpenStreetMap (OSM) or Google Maps Platform APIs.

Figure below illustrates an example of geometry objects stored in AWS RDS PostgreSQL+ PostGIS extension:

Buildings		
building_id	building_name	geometry
B001	City Hall	Polygon(103.8198 1.3521)
B002	Hospital	Polygon(224.8198 1.3321)
Roads		
road_id	road_name	geometry
R001	Main Street	LineString(106.8198 1.3521)
R002	River Road	LineString(108.1238 1.3421)
River		
river_id	river_name	geometry
RV001	Kallang River	LineString(108.1238 1.3421)
Flood Zones		
zone_id	risk_level	geometry
F001	High	Polygon(102.8198 1.3521)
F002	Medium	Polygon(204.8198 1.3321)

AI Prediction Engine Data

The AI prediction model will predict flood probabilities and severity level using real time data collected from IoT sensors and weather forecast data obtained from NEA Weather Forecast API. It generates flood prediction and impact analysis reports and stores them into AWS Dynamo DB, a NoSQL key-value and document database. Each report is stored as a document containing key-value pair attributes.

The generated reports are subsequently transmitted to downstream agencies, including the Singapore Police Force (SPF), Singapore Civil Defence Force (SCDF), and Town Councils, via REST APIs. This enables stakeholders to receive timely situational awareness, support informed decision-making, improve emergency preparedness, and coordinate response efforts to enhance public safety.

Below illustrates an example of document objects stored in AWS Dynamo DB:

{
"reportId": "RPT001",
"timestamp": "2026-07-26T10:30:00Z",
"location": "Ion Orchard Shopping Mall",
"floodProbability": 0.87,
"severityLevel": "High",
"predictedWaterLevel": 3.8,
"affectedAreas": [
"Zone A",
"Zone B"
],
"recommendedActions": [
"Issue flood warning",
"Close nearby roads"
]
}

6.1.2 Dataset Query Processing

Data stored are queried to retrieve information for analysis and decision making. Different types of data require different query languages, depending on the type of database in which they are stored.

6.1.2.1 Weather Forecast Query

2-Hour Weather Forecast Data retrieved from NEA REST API stored in AWS ElastiCache (Redis) are queried using Redis Command Language (RESP protocol). Every record inserted is automatically indexed making lookups extremely fast. The data queried provides current weather conditions for AI prediction.

Example query: Latest 2-hour weather forecast for Tampines, Pasir Ris, Jurong East, Woodlands.

6.1.2.2 Historical Weather Forecast Query

2-Hour Weather Forecast Data retrieved from NEA REST API stored in AWS RDS PostgreSQL are queried using SQL language. Indexing scripts are executed during creation of relational tables to enable fast lookup. The data queried provides historical weather records for data analysis, trend detection, model training, reporting and also auditing.

Example query: 2-hour weather forecast records for the last 5 years

6.1.2.3 Time-Series Query (IoT Devices)

Real-time water level, flow rate and rainfall intensity data retrieved from sensor devices stored in AWS Timestream are queried using SQL language as it supports SQL syntax. However, unlike

traditional relational databases, records inserted are automatically indexed. The data queried provides real-time flood monitoring and early flood prediction.

Example query: Current water level, rainfall intensity and flow rate readings

6.1.2.4 GIS Spatial Query (IoT Device Location)

IoT sensors' location data are stored in AWS RDS PostgreSQL with PostGIS extension enabled area queried using SQL language. With PostGIS extension enabled, it supports storing geometry objects in addition to normal relational data. Spatial indexes (GiST Generalized Search Tree) are created during creation of relational tables. This helps to avoid scanning every sensor when using queries like nearest-neighbour to ensure fast lookup. The data queried provides sensor location within a certain distance of a predicted flood.

Example query: All sensors closest to the predicted flood location of Newton MRT station

6.1.2.5 AI Prediction Query

Data from weather forecasts, live sensor readings and historical data are feeded to the AI prediction engine to generate flood predictions. It is the core function of proactive flood prediction and risk analysis.

Example query: List all predicted flood probability and severity

6.1.2.6 GIS Spatial Query (Mapping Overlay)

When the AI prediction engine flags all predicted flood locations, it sends flood location objects (polygons) to the GIS database for spatial querying. PostgreSQL with PostGIS then performs spatial overlay analysis to identify roads, buildings, and other infrastructure that intersect the predicted flood location. This enables flood impact assessment and damage control.

Example query: All infrastructures that intersect the predicted flood area of Newton MRT station

6.1.2.7 Document Query

Flood prediction reports generated by the AI prediction engine are stored in AWS DynamoDB. These reports are distributed to downstream agencies, such as SCDF, SPF, and Town Councils to support situational awareness and emergency response. The reports can also be retrieved for auditing, reporting, and survey purposes.

Example query: All flood prediction reports for Bugis generated within the last 2 weeks

6.1.2.8 Analytics Features

S/N	Analytics Type	Purpose	Data Source
1	Historical	Historical weather conditions for data analysis, trend detection, model training, reporting and also auditing.	NEA REST API data
2	Descriptive	Monitors current weather conditions, rainfall, water level and flow rate across Singapore.	NEA REST API data, IoT Devices (Sensors)
3	Predictive	Forecasts flood probability and severity using AI prediction models.	NEA REST API data, IoT Devices (Sensors), GIS Dataset
4	Spatial	Map surrounding infrastructure that overlay with predicted flood location to perform impact analysis and damage control.	AI Prediction Engine, GIS Dataset
5	Prescriptive	Recommends response actions based on predicted flood impact such as alternative road routes, road closure and public warning.	AI Prediction Engine

6.1.3 Software Engineering

Figure 6 illustrates the architectural diagram for this function (AI-Based Flood Prediction & Risk Analysis):

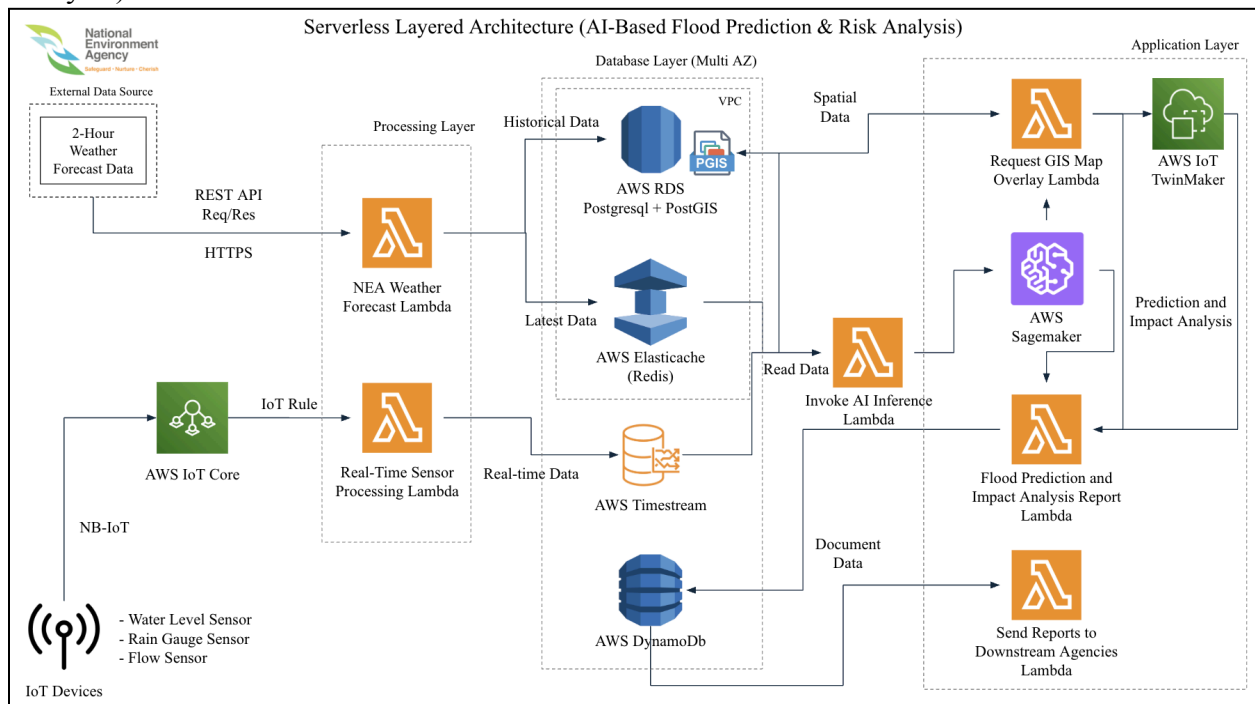


Figure 6. Architectural Diagram for AI-Based Flood Prediction & Risk Analysis

This function (AI-Based Flood Prediction & Risk Analysis) adopts a serverless layered architecture hosted and deployed in AWS Cloud Ecosystem.

- Serverless (No hosting of servers (EC2), AWS manages the servers)
- Layered (Divide systems into separate layers with separate responsibilities)

Data Source

There are 2 main types of data sources:

- NEA Weather Forecast Data
- IoT Sensor Readings Data

The NEA weather forecast data is transmitted from NEA's endpoint to FloodWatch's endpoint via a (Representational State Transfer) REST API. To establish successful data transmission, the NEA's endpoint IP address must first be whitelisted as FloodWatch endpoint only accepts connections from authorised IP addresses.

The IoT sensor readings data are authenticated and verified by AWS IoT Core, a service that routes IoT devices' data to the processing layer.

Processing Layer

In the processing layer, AWS Lambda processes the incoming NEA weather forecast data and IoT sensor readings by validating them before they are ingested into the databases.

- AWS Lambda is a serverless function that executes code when triggered by an event
- Data Transformation is the process of cleaning and restructuring data to a suitable format before data insertion to databases
- Data Ingestion is the process of importing data from source to destination

Database Layer

The database layer is responsible for storing, managing and retrieving processed data for analysis, reporting, AI model training and prediction. Multi-Available Zone (AZ) is enabled to ensure AWS performs automatic failover to the standby AZ should the primary AZ fail. This helps reduce risks of downtime.

Multiple databases are used for this function:

S/N	Data	Database	Database Type
1	2-Hour Weather Forecast	AWS RDS Postgres SQL	SQL
2	2-Hour Weather Forecast	AWS ElastiCache (Redis)	NoSQL (Cache)
3	Water Level Sensor, Flow Sensor, Rain Sensor	AWS Timestream	Time-Series DB

4	Sensor Location	AWS RDS PostgreSQL + PostGIS	Spatial DB
5	GIS Data (Buildings, Roads, Rivers, Flood Zones)	AWS RDS PostgreSQL + PostGIS	Spatial DB
6	Flood Prediction Report and Incident Report	AWS DynamoDB	Document DB

Application Layer

Each time new sensor readings or updated NEA weather forecast data have been processed and stored in their respective databases, the corresponding processing Lambda function invokes the AI Inference Lambda.

The AI Inference Lambda is responsible for retrieving the latest real-time sensor readings from Amazon Timestream, the latest weather forecast from Amazon ElastiCache (Redis), and historical flood and spatial data from Amazon RDS PostgreSQL. It then combines the raw data from different sources (Aggregation) and formats it into the input required by the ML model.

The prepared input is then sent to AWS SageMaker inference endpoint in the application layer where the AI ML model generates flood prediction and performs impact analysis. For each predicted flood event, the AI prediction engine identifies predicted flood locations and passes the data to the GIS Overlay Map Lambda.

The Request GIS Map Overlay Lambda queries the PostGIS database to retrieve spatial data and performs spatial overlay analysis by intersecting the predicted flood locations with roads, buildings, rivers, and other infrastructure. This identifies the infrastructure that is potentially affected by the predicted flood event.

The spatial analysis results are then passed to the Flood Prediction and Impact Analysis Report Lambda which generates a structured flood prediction and impact analysis report and stores it in Amazon DynamoDB.

Parallely, spatial analysis results are passed to AWS IoT TwinMaker, which creates a real-time representation of a physical infrastructure. This provides PUB users with a live, interactive visualization of Singapore's drainage network, real-time flood conditions, and AI-predicted flood hotspots, enabling faster situational awareness and more informed operational decision-making.

Finally, the Send Reports to Downstream Agencies Lambda retrieves the generated report from Amazon DynamoDB and distributes it to the relevant downstream agencies to support situational awareness, emergency response, and operational decision-making.

AI Application

The core functionality of this feature relies on the use of AI to predict flooding events. AWS SageMaker is a platform that provides AI models with the tools to build, train, tune, deploy and manage the entire AI workflow. The aggregated raw data prepared by the AI Inference Lambda is processed in AWS SageMaker where processing jobs, data wrangler and python libraries (NumPy) are used to clean and transform the data before it is passed to an AI model.

The AI models used are:

S/N	AI Model	Purpose
1	Long Short-Term Memory (LSTM)	To predict future water levels
2	Random Forest	To determine flood risk levels

Each AI model handles a different part of the prediction problem.

1. Long Short-Term Memory (LSTM)

LSTM is a type of Recurrent Neural Network (RNN) designed for learning and forecasting sequential (time-series) data. It is well suited for water level prediction as water levels are continuous real-time measurements that depend on historical observations and temporal patterns in water levels, rainfall, water flow and weather conditions.

The LSTM model uses both real-time data and historical data as inputs including:

- Historical water levels (Previous water level measurements)
- Rainfall data (Current and past rainfall intensity)
- Water flow (Volume and speed of water flowing through rivers or drainage systems)
- Weather forecast (Predicted rainfall, temperature)

These variables serve as inputs to the LSTM model and which predicts the future water level as its output. It is then passed to the Random Forest model.

2. Random Forest

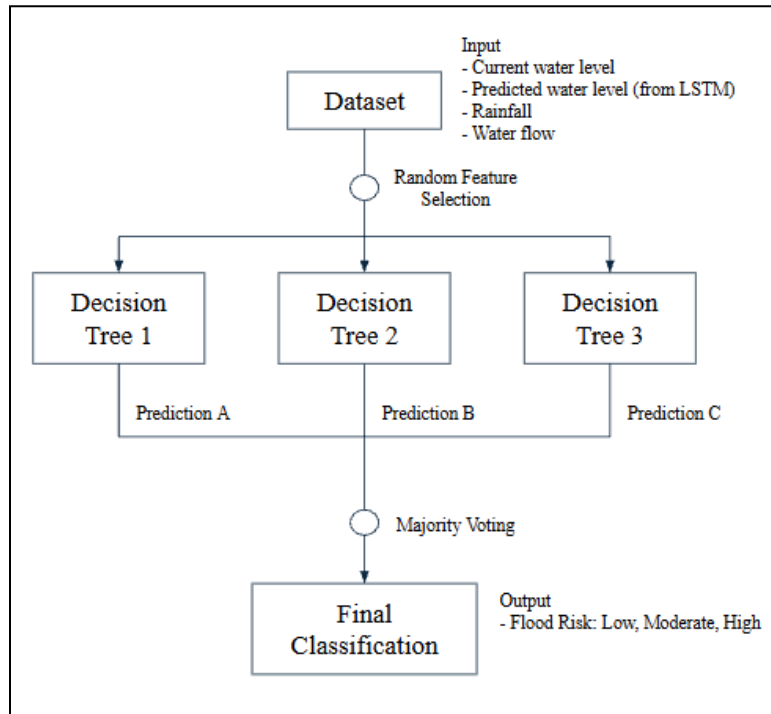
Random Forest is a machine learning classification algorithm that consists of multiple decision trees. Each decision tree independently makes a prediction and the Random Forest combines all the tree's predictions using a majority voting mechanism to produce a final result. It is a classifier to determine flood risk levels by using predicted water levels from LSTM with other environmental variables to classify the flood risk into categories (Low, Moderate, High).

For example:

- If predicted water level < 2.5m (from LSTM), low rainfall and short duration → Low flood risk
- If predicted water level > 2.5m (from LSTM), low rainfall and short duration → Moderate flood risk

- If predicted water level > 2.5m (from LSTM), high rainfall and long duration → High flood risk

Figure below illustrates the workflow and classified dataset of Random Forest:



Current Water Level	Predicted Water Level (LSTM)	Rainfall	Duration	Flood Status
1.4m	1.8m	5mm	Short	Low
1.4m	2.6m	25mm	Short	Moderate
1.4m	2.9m	40mm	Long	High

Once the data is finalised and classified into a flood risk level (low, moderate, high), the Request GIS Overlay Lambda is invoked to retrieve and overlay GIS spatial data for the predicted impacted area.

In summary, LSTM and Random Forest work together to provide accurate and timely flood predictions:

- LSTM analyses historical and real-time time-series data to forecast future water levels
- Random Forest classifies the predicted flood severity based on multiple environmental variables

By combining both models, the system can proactively identify potential flood events, assess their severity, and support early warning and emergency response efforts.

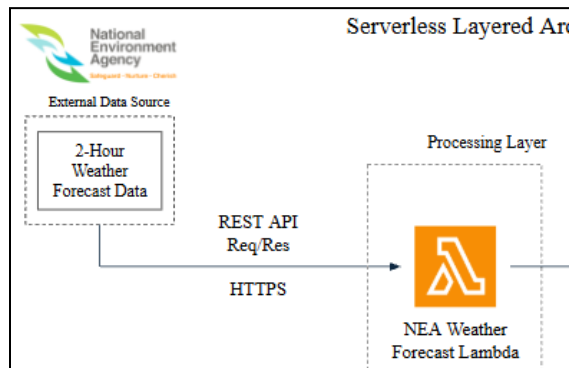
6.1.4 Networking

Various types of IoT Sensor devices to be deployed across Singapore:

- Water Level Sensor
 - Measures the height level of water. They are installed alongside canals, drains and reservoirs
- Flow Sensor
 - Measures the water velocity in canals, drains, rivers and reservoirs. They are installed in waterpipes
- Rain Gauge Sensor
 - Measures the amount and intensity of rainfall. They are installed in open outdoor areas and rooftops

Source	Destination	Communication Protocol / Rule
NEA Weather Forecast	AWS Weather Forecast Lambda	HTTPS
IoT Devices	AWS IoT Core	NB-IoT
AWS IoT Core	AWS Real-Time Sensor Processing Lambda	IoT Rule

NEA Weather Forecast to AWS Weather Forecast Lambda



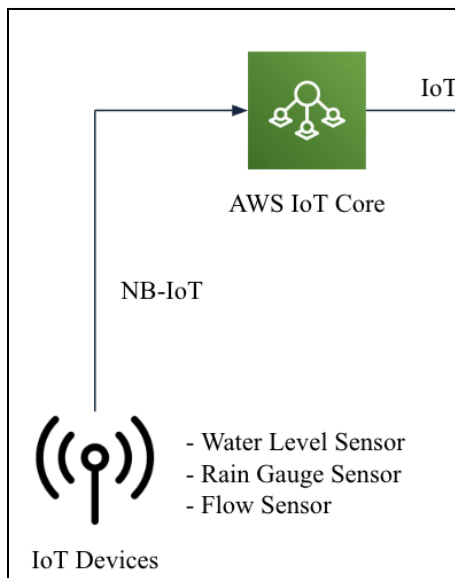
The NEA weather forecast data (payload) is transmitted from NEA's endpoint to FloodWatch's endpoint via a REST API

- Payload is the data sent between client and server
- An API (Application Programming Interface) is an interface that allows software applications to communicate with one another
- REST (Representational State Transfer) is an architectural style for designing web APIs with HTTP methods (GET, POST, PUT, DELETE)

The communication protocol used is HTTPS (HTTP + TLS)

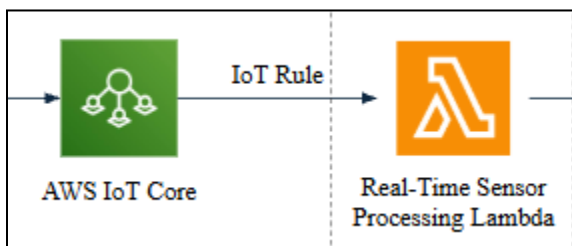
- Hyper Text Transfer Protocol (HTTP) are used to exchange payload between client and server over a network
- Payloads exchanged over HTTP are unencrypted by default and can be intercepted or modified by a hacker. Hence, Transport Layer Security (TLS) are used to encrypt the payload before transmitting over the network

IoT Devices to AWS IoT Core



Real-time sensor data is transmitted from the IoT Devices used in this function to AWS IoT Core using NB-IoT (Narrowband Internet of Things) connectivity. Just like a smartphone, each IoT device is equipped with a SIM card, allowing it to connect to the cellular network and transmit sensor data to AWS IoT Core.

AWS IoT Core to AWS Real-Time Sensor Processing Lambda



Once data is received by AWS IoT Core from IoT devices, it is evaluated against the configured IoT Rules before invoking AWS Real-Time Sensor Processing Lambda.

- If data does not match the criteria, the data is not forwarded to the lambda
- If data matches the criteria, the data is accepted and the processing lambda is invoked

6.1.5 Advanced Technologies and Security

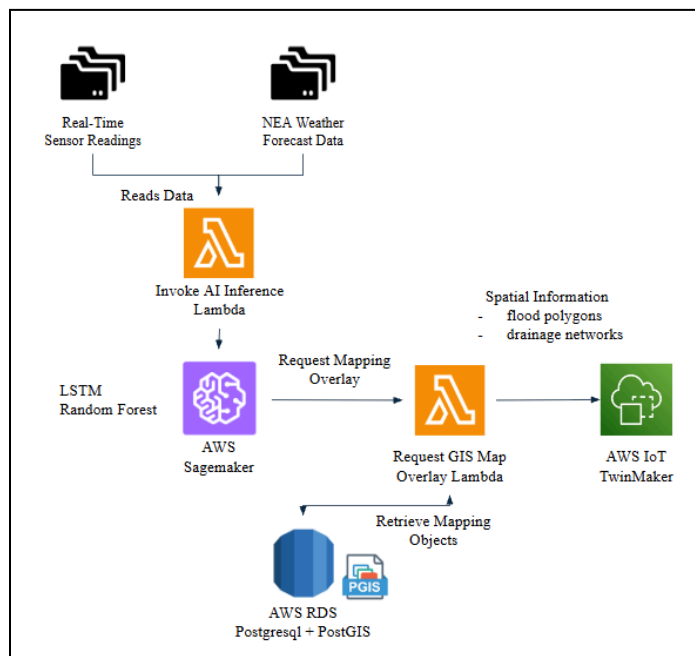
Digital Twin

This feature also incorporates digital twin technology to represent the physical infrastructures such as roads, rivers and drainage canals. The spatial analysis results are integrated with AWS IoT TwinMaker, it creates a virtual representation of Singapore's drainage network and infrastructure such as buildings and roads allowing PUB users to monitor situations in real-time.

The digital twin is built by adopting the Digital Twin Model (Data → Model → Software) integrating multiple data sources (Data), including real-time sensor data, GIS mapping, and AI-generated prediction results. While AI models perform the flood prediction and spatial analysis (Model), the digital twin itself does not carry out AI prediction. Instead, it serves as a visualization and monitoring platform that consolidates information from these sources into a single, interactive virtual representation of the physical environment (Software).

This enables PUB to observe current conditions, conduct impact analysis, and improve situational awareness, which supports more effective flood management and emergency response.

The figure below illustrates the data flow within AWS IoT TwinMaker for creating digital entities that represent physical infrastructure.



Physical Object	Digital Twin Entity
Drainage Canal A	Canal_A

Yishun MRT Station	Yishun_MRT_Station
Flood Zone	Flood_Zone_1

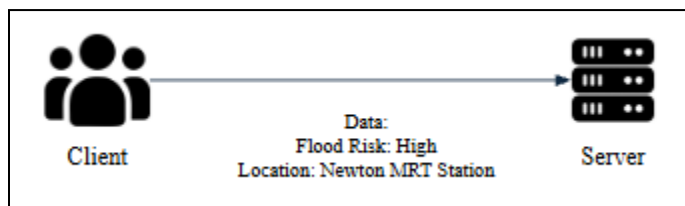
Cross Origin Resource Sharing (CORS)

In order to establish successful data transmission, the NEA's endpoint IP address must first be whitelisted. FloodWatch endpoint only accepts connections from authorised IP addresses preventing unknown sources from connecting to FloodWatch enhancing security of data exchange.

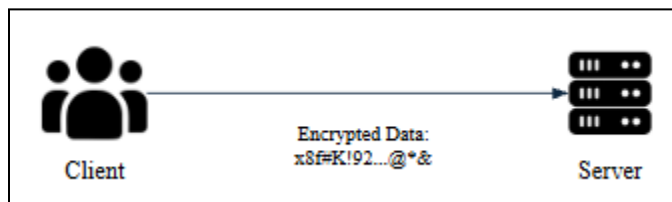
Transport Layer Security (TLS)

Formerly known as Secure Socket Layer (SSL) which is a security protocol that encrypts data exchanged between client and server.

Without TLS:



With TLS:



In this feature, the NEA Weather Forecast Lambda sends a HTTPS request connection to NEA Weather API, the NEA Server sends its TLS Certificate. The lambda verifies its origin to ensure it is issued by a trusted Certificate Authority (CA) and not expired and matches the NEA domain.

Once the certificate is successfully verified, the lambda and NEA establish a secure connection with a shared session key. NEA server encrypts the payload using the same session key and sends the payload to the lambda. Once the data transmission is completed, the TLS session is terminated.

6.2 Drainage & Pump Health Monitoring

The Drainage & Pump Health Monitoring function is designed to continuously monitor the operational status and structural condition of Singapore's drainage infrastructure, including drains, canals, culverts, and pumping stations, to ensure they remain capable of effectively conveying stormwater during periods of heavy rainfall. Although Singapore has invested extensively in its drainage network to mitigate flash flooding, the performance of this infrastructure can deteriorate over time due to debris accumulation, sediment build-up, mechanical wear, and unexpected equipment failure. Left undetected, these issues progressively reduce drainage capacity, impede water flow, and increase the risk of flash flooding, particularly during short-duration, high-intensity rainfall events.

To address this, the function adopts a proactive, sensor-driven approach that combines Internet of Things (IoT) sensing, edge computing, and a digital twin representation of Singapore's drainage assets. IoT sensors continuously collect operational data such as water levels, flow rates, pump vibration, motor temperature, and electrical current consumption from drainage assets and pumping stations across Singapore. Rather than relying on centralised machine learning models, the function applies deterministic, rule-based complex event processing to continuously compare incoming readings against calibrated engineering thresholds and rolling statistical baselines, flagging abnormal operating patterns as soon as they occur. This lightweight approach keeps detection transparent and explainable to PUB engineers, and fast enough to react to sudden mechanical faults such as a stalled pump motor.

Each monitored asset is additionally mirrored by a live digital twin that consolidates its sensor readings, maintenance history, and calculated health status into a single virtual representation. This allows engineers to visually inspect the real-time condition of a drain, canal, or pump remotely, run simple “what-if” capacity checks against predicted rainfall, and prioritise maintenance without needing to be physically present at the asset. Maintenance events and inspection outcomes are further recorded to an immutable ledger, giving PUB and its maintenance contractors a tamper-evident audit trail of every intervention performed on critical infrastructure.

6.2.1 Significance

Effective drainage infrastructure is a fundamental component of Singapore's flood management strategy, particularly given the large proportion of impermeable surfaces that increase surface runoff during heavy rainfall. While extreme rainfall is a major contributor to flash flooding, the operational reliability of the drainage network itself is equally important in determining whether stormwater can be conveyed away from flood-prone areas. Even under moderate rainfall, a blocked drain, sediment build-up, or a malfunctioning pump can substantially reduce drainage capacity, resulting in localised flooding, traffic disruption, and safety risks to the public.

This function directly supports FloodWatch's objective of transforming Singapore's flood management approach from reactive response to proactive prevention. By continuously assessing infrastructure health and surfacing abnormal conditions before they escalate into failures, maintenance teams can intervene early, reducing emergency repair costs, minimising service disruption, and extending the operational lifespan of critical drainage assets.

The infrastructure health information generated here also complements the AI-Based Flood Prediction & Risk Analysis function: while that function forecasts flood probability and severity based on meteorological and hydrological conditions, this function evaluates whether the physical drainage infrastructure has sufficient operational capacity to cope with the predicted rainfall. Combining environmental forecasts with infrastructure readiness allows FloodWatch to produce a more complete flood risk picture, enabling PUB to prioritise maintenance, allocate resources, and strengthen Singapore's overall resilience against increasingly frequent flash flood events.

6.2.2 Multi-Modal Database

This function reuses the same multi-modal database architecture as the AI-Based Flood Prediction & Risk Analysis function to ensure seamless integration within the FloodWatch platform. While the flood prediction module primarily manages weather and hydrological data, this function stores and analyses infrastructure health data generated by drainage sensors and pumping stations, together with the maintenance ledger. Four database models are used, each suited to a different data type.

6.2.2.1 Time-Series Database (Amazon Timestream)

Amazon Timestream stores high-frequency IoT sensor readings collected from drainage infrastructure, including water levels, flow rates, pump vibration, motor temperature, and electrical current. As these measurements are generated continuously, a time-series database enables efficient storage, real-time querying, and rolling statistical baseline calculation (e.g. moving averages and standard-deviation bands), which the rule-based complex event processing engine uses to flag abnormal operating conditions and estimate pump failure risk.

Example query: SQL

```
SELECT AVG(vibration), STDDEV(vibration)
FROM PumpSensorData
WHERE pump_id = 'P001'
AND time > NOW() - INTERVAL '24 HOURS';
```

6.2.2.2 Document Database (Amazon DynamoDB)

Amazon DynamoDB stores semi-structured operational information such as maintenance reports, inspection records, rule-engine-generated recommendations, and fault histories. The flexible

document model allows maintenance information to evolve without requiring schema modifications, making it suitable for recording diverse maintenance activities performed across different drainage assets.

Example document: JSON

```
{  
  "pumpId": "P001",  
  "fault": "Bearing Wear",  
  "riskLevel": "High",  
  "recommendedAction": "Replace Bearing",  
  "status": "Pending"  
}
```

6.2.2.3 Spatial Database (PostgreSQL + PostGIS)

Similarly to the flood prediction function, PostgreSQL with the PostGIS extension manages geospatial information within the FloodWatch platform. Besides storing flood-prone areas and weather station locations, the database also records the geographical locations of drains, canals, pumping stations, and maintenance zones, allowing engineers to identify drainage assets located within predicted flood-risk areas and prioritise inspection or maintenance activities.

Example query: SQL

```
SELECT pump_name  
FROM pump_station  
WHERE ST_DWithin(  
  pump_location,  
  predicted_flood_zone,  
  500  
);
```

6.2.2.4 Ledger Database (Amazon QLDB)

Amazon QLDB provides an immutable, cryptographically verifiable ledger that records every maintenance action, inspection outcome, and automated work order raised against a drainage asset. Because entries can only be appended and never altered or deleted, PUB and its maintenance contractors gain a transparent, tamper-evident audit trail that supports regulatory compliance, dispute resolution, and long-term reliability reporting.

Example query: PartiQL

```
SELECT * FROM MaintenanceLedger  
WHERE assetId = 'PUMP-014'  
AND eventType = 'WORK_ORDER_ISSUED';
```

6.2.3 Networking Infrastructure

This function extends the same networking backbone used by FloodWatch to ensure interoperability across the FloodWatch platform. Additional sensing devices, including water level, flow, vibration, motor temperature, and electrical current sensors, are deployed across Singapore's drainage channels, canals, and pumping stations.

All sensors publish telemetry using the lightweight MQTT protocol over NB-IoT (Narrowband IoT), a low-power cellular technology, to AWS IoT Core, the central IoT gateway responsible for device authentication and secure message routing. NB-IoT's deeper signal penetration and lower power consumption make it well suited to drainage assets such as underground culverts and enclosed pump chambers, where conventional cellular coverage is often poor, while its narrowband design is a good fit for the small, infrequent sensor readings this function transmits. Using a single network technology for both above-ground and underground sensors keeps the deployment consistent and avoids managing two separate radio infrastructures, and matches the NB-IoT connectivity used elsewhere in FloodWatch, ensuring the same sensor types operate on one unified network architecture regardless of which function deployed them.

Each sensor node additionally performs lightweight edge processing before transmission: local firmware pre-aggregates raw readings (e.g. 1-second samples into 1-minute summaries) and evaluates them against locally cached threshold bands, allowing time-critical local alerts, such as an abrupt vibration spike, to be raised even during a temporary loss of connectivity. Once connectivity is restored, buffered readings are forwarded to AWS IoT Core and Amazon Kinesis Data Streams for centralised processing.

All communications are protected using Transport Layer Security (TLS) encryption and AWS IoT X.509 certificate-based authentication, ensuring that only authorised devices can communicate with the FloodWatch platform, and preserving data confidentiality, integrity, and system reliability.

6.2.4 Software Architecture

This function adopts the same serverless, cloud-native architecture used in FloodWatch, extended with a digital twin and ledger-backed maintenance workflow, as illustrated in Figure 7.

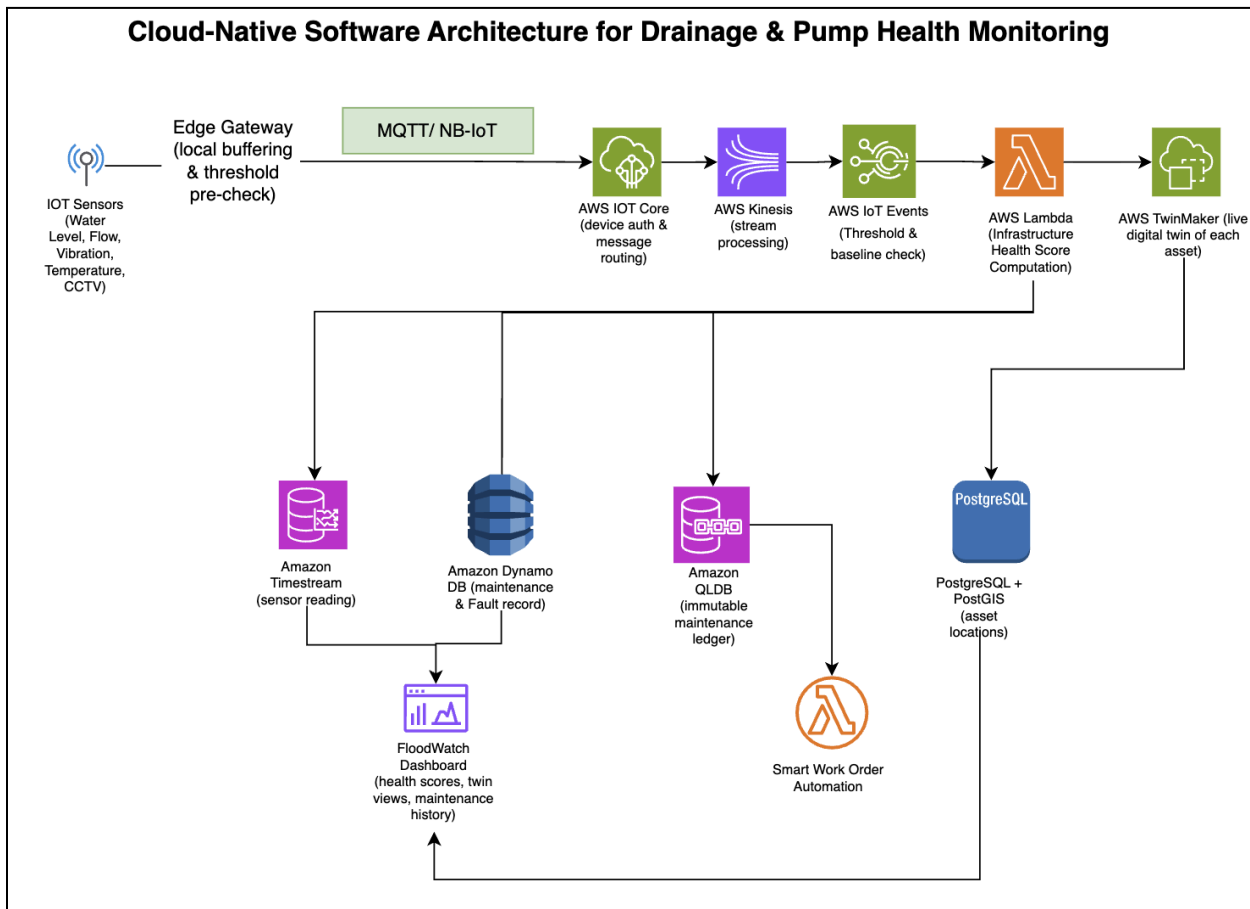


Figure 7. Architecture diagram of the Drainage & Pump Health Monitoring function.

6.2.4.1 Data Acquisition Layer

Operational data collected from water level, flow, vibration, motor temperature, and electrical current sensors are transmitted, via the networking paths described above, to AWS IoT Core, which authenticates each device and forwards validated telemetry into Amazon Kinesis Data Streams for real-time ingestion.

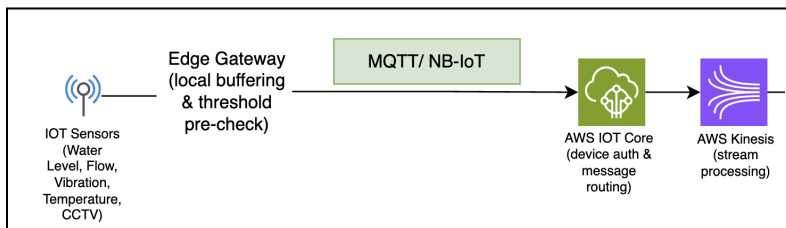


Figure 8. Data Acquisition Layer

6.2.4.2 Processing Layer

Incoming telemetry is evaluated by AWS IoT Events, a managed complex event processing (CEP) service that continuously compares live readings against calibrated engineering thresholds and rolling statistical baselines stored in Amazon Timestream. When a reading breaches its threshold band, for example a sustained rise in pump vibration or motor temperature, AWS IoT Events raises a detector state change that triggers an AWS Lambda function to compute an Infrastructure Health Score for the affected asset and determine its likely fault category.

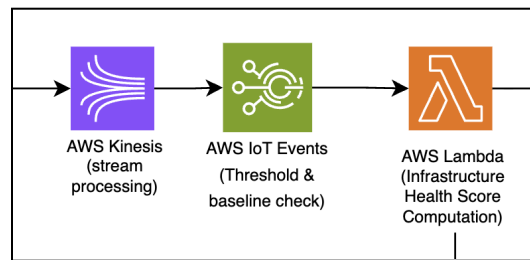


Figure 9. Processing Layer

6.2.4.3 Digital Twin Layer

The computed health status, together with the asset's latest sensor readings, is pushed to AWS IoT TwinMaker, which maintains a live digital twin for every monitored drain, canal, and pumping station. Each twin links its virtual model to real-time and historical data sources, allowing PUB engineers to visually inspect the current condition of an asset, review its recent sensor trend, and simulate the impact of a predicted rainfall event on its available capacity, without needing to visit the site physically.

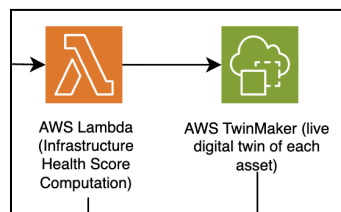


Figure 10. Digital Twin Layer

6.2.4.4 Database Layer

Structured sensor readings are stored in Amazon Timestream, semi-structured maintenance and fault records in Amazon DynamoDB, geospatial asset locations in PostgreSQL with PostGIS, and maintenance audit records in Amazon QLDB, as described in Section 6.2.2.

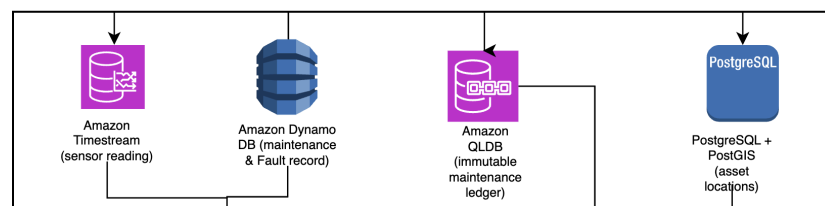


Figure 11. Database Layer

6.2.4.5 Application Layer

When an asset's Infrastructure Health Score falls below a configured threshold, a smart-contract-style rule executes automatically: an AWS Lambda function generates a maintenance work order, writes an immutable record of the triggering condition to Amazon QLDB, and dispatches the work order to PUB's maintenance team through the FloodWatch Dashboard. This automated, condition-based workflow shortens the time between fault detection and maintenance action, without requiring an engineer to manually review every sensor alert. The FloodWatch Dashboard itself presents the Infrastructure Health Score, live digital twin visualisations, and maintenance history for every monitored asset, enabling PUB engineers to prioritise preventive maintenance and track resolution of outstanding work orders.

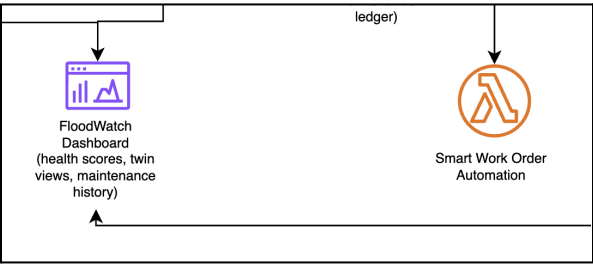


Figure 12. Application Layer

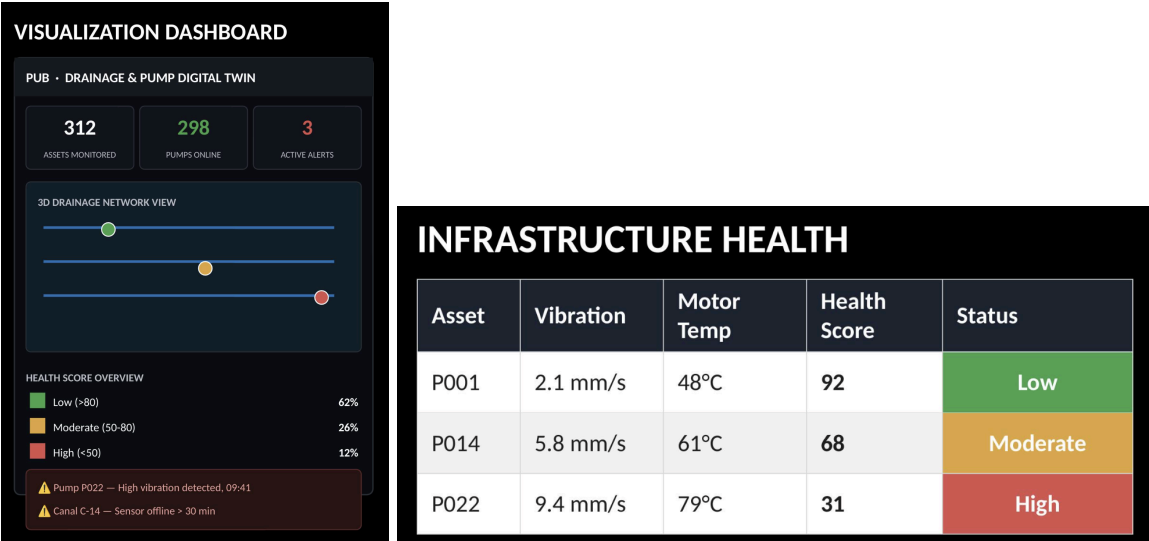


Figure 13 & 14. Digital Twin Visualization Dashboard & Infrastructure Health Score Meter

6.3 Early Warning & Multi-Channel Alert Dissemination

6.3.1 Functional Overview

The purpose of an Early Warning & Multi-Channel Alert Dissemination function is to generate and disseminate early warnings the moment a flood risk is identified, whether if it is from a direct sensor threshold or from the AI-Based Flood Prediction & Risk Analysis engine, ensuring the right stakeholders are warned through the right channel within seconds is essential. Rather than identifying threats, this function determines who needs to be warned in real time, which communication channel should be used, and whether the warning was successfully received. It converts a raw detection or forecast into an actual early warning reaching people who can act on it, and it is deliberately kept lightweight and rule-based rather than model-heavy, so that the warnings continue to go out promptly even if the predictive layer is degraded or delayed.

Early warnings are generated from 2 triggers running in parallel:

1. A direct threshold breach on a live sensor reading, such as water level in a canal crossing a calibrated depth percentage, giving an immediate, prediction-independent warning.
2. A new high-risk flood prediction report generated by the AI Prediction Engine in **1. AI-Based Flood Prediction & Risk Analysis**, forward-looking warning before a physical threshold is even reached.

Either trigger causes this function to score the warning's severity, determine the geographic area and cell towers it should reach, and disseminate it to stakeholders via telco cell broadcast and the FloodWatch app, and to agencies via structured, acknowledgement-tracked channels. This closes the loop between warning and action for agencies and app users, addressing a key limitation of many conventional flood alerting systems, where they broadcast an alert but have no mechanism to confirm it was received or to escalate if it was not acknowledged.

6.3.2 Multi-Modal Database

6.3.2.1 Dataset Storage

The Early Warning & Multi-Channel Alert Dissemination function reads 2 datasets it does not own, which are live sensor readings and flood prediction reports from **6.1 AI-Based Flood Prediction & Risk Analysis**, purely as warning triggers and owns 7 datasets of its own that support warning configuration and dissemination, spanning relational, document and ledger database types, as shown below.

S/N	Data	Update Frequency	Data Source	Data Destination	Database Type
1.	Warning Threshold	On config	PUB/Syste	AWS RDS	SQL

	Configuration (per sensor/ flood zone)	change	m Admin	PostgreSQL	
2.	Agency Endpoint & Escalation Contact Registry (PUB, SCDF, SPF, LTA, Town Council systems and on-call contacts)	On registration / update	Agency Onboarding	AWS RDS PostgreSQL	SQL
3.	Public App Subscription Registry (opt-in, app tokens + subscribed zones, for residents who install the app)	On registration/ update	Public App	AWS RDS PostgreSQL	SQL
4.	Flood Zone to Cell Broadcast Geofence Mapping (zone boundary to target cell tower/broadcast area)	As required	PUB/Telco Coordination	AWS RDS PostgreSQL + PostGIS	Spatial DB
5.	Live Sensor Readings (consumed, read-only)	Real-Time	AWS Timestream (shared with prediction engine)	Warning Decision Lambda	Time-Seri es DB
6.	Flood Prediction & Risk Reports (consumed, read-only)	Event-driven	AWS DynamoDB (shared, from AI Prediction Engine)	Warning Decision Lambda	Document DB
7.	Warning Dispatch Log (channel, recipient, severity, timestamp, status)	Event-driven	Warning Decision Lambda	AWS DynamoDB	Document DB (NoSQL)
8.	Delivery & Acknowledgement Status (agency, siren/signage & opt-in app channels)	Near real-time (on callback)	Agency API/IoT Core/App Callbacks	AWS DynamoDB	Document DB (NoSQL)
9.	Warning Audit Record (immutable, tamper-evident copy of dispatch & acknowledgement events)	Event-driven	Warning Decision Lambda/ Escalation Manager	Amazon QLDB (ledger)	Ledger DB

Figure 15. Dataset storage for the Early Warning & Multi-Channel Alert Dissemination function

6.3.2.2 Dataset Query Processing

Threshold Configuration Query

Warning threshold configuration data stored in AWS RDS PostgreSQL are queried using SQL. An index on sensor ID and flood zone is created when the table is set up, so the Warning Decision Lambda can look up the correct threshold in milliseconds every time a new sensor reading arrives. The data queried provides the calibrated depth or severity threshold a live reading is compared against.

Example Query: Retrieve the active alert threshold configuration for all sensors in the Bugis flood zone.

Agency Endpoint & Escalation Contact Query

Agency Endpoint and on-call contact data stored in AWS RDS PostgreSQL are queried using SQL, indexed by flood zone and severity tier. This lets the Warning Decision Lambda quickly resolve which agency systems and escalation contacts should receive a warning for a given zone and severity level.

Example Query: Retrieve all agency endpoints and on-call contacts registered for High-severity alerts affecting Tampines.

Cell Broadcast Geofence Query

Flood zone to cell tower mapping data stored in AWS RDS PostgreSQL with the PostGIS extension are queried using SQL with spatial functions such as ST_intersects. A spatial index (GIST) on the geometry column allows the system to find every cell tower overlapping a flood zone polygon without scanning the full table. The data queried defines exactly which cell towers a public cell broadcast should be sent through.

Example Query: Determine the target cell towers covering the predicted flood zone at Newton MRT station.

Time-Series Query (Consumed, read-only)

Real-time water level, flow and rainfall readings stored in AWS Timestream (populated by the AI-Based Flood Prediction & Risk Analysis function) are queried using Timestream's built-in SQL support. This function only reads from this dataset, using the latest readings to evaluate whether a threshold has been breached.

Example Query: Latest water level and rainfall intensity readings for sensors within 500m of Newton Mrt Station.

Document Query (Prediction reports, consumed)

Flood prediction reports stored in AWS DynamoDB (populated by the AI Prediction Engine) are queried using DynamoDB's key-value lookup and attribute filters, indexed by severity and

timestamp. This function reads these reports as a second, forward-looking warning trigger alongside live sensor thresholds.

Example Query: All flood prediction reports with severity “High” generated in the last 15mins.

Document Query (Warning Dispatch Log)

Dispatch log entries stored in AWS DynamoDB are queried using DynamoDB’s document query API, indexed by alert ID, zone and recipient. The data queried supports operational reporting on which warnings were sent, when, and through which channel.

Example Query: All warnings dispatched to SCDF for the Bugis flood zone in the last 24 hours.

Document Query (Delivery & Acknowledgement Status)

Delivery and acknowledgement records stored in AWS DynamoDB are queried using DynamoDB’s document query API, filtered by acknowledgement status and elapsed time. The data queried identifies warnings that remain unacknowledged, which directly feeds the Escalation Manager’s retry and escalation decisions.

Example Query: All warnings with no acknowledgement received more than 3 minutes after dispatch.

Public App Subscription Query

Public app subscription data stored in AWS RDS PostgreSQL are queried using SQL, indexed by user ID and subscribed zones. This supports the optional richer push channel for residents who have chosen to install the app, on top of the mandatory telco cell broadcast every phone already receives.

Example Query: Retrieve all app users subscribed to alerts for the Bugis flood zone.

Ledger Query (Warning Audit Record)

Audit records stored in Amazon QLDB are queried using PartiQL, a SQL-compatible query language for ledger databases that also supports verifying an entry’s cryptographic change history. The data queried supports post-incident review and accountability audits across agencies.

Example Query: Verify the dispatch and acknowledgement history for alert ID FW-20260803-0142.

6.3.2.3 Analytics Features

S/N	Analytics Type	Purpose	Data Source
1.	Descriptive	Monitors current alert dispatch status	Warning Dispatch Log

		and channel-by-channel delivery volumes across the island.	(DynamoDB)
2.	Diagnostic	Identifies which stakeholders, zones or channels have the highest delivery-failure or non-acknowledgement rates.	Warning Dispatch Log, Delivery/Acknowledgement Status
3.	Real-Time/Operational	Triggers an alert the instant a sensor threshold is breached or a new high-risk prediction report is received.	AWS Timestream, DynamoDB (prediction reports)
4.	Historical	Supports audit, reporting and after-action review of past alert dispatches and response times.	Warning Dispatch Log, Amazon QLDB
5.	Prescriptive	Recommends escalation actions (e.g. re-dispatch, escalate to next stakeholder tier, activate physical sirens) when an alert is not acknowledged within a defined time window.	Escalation Manager (Step Functions)

Figure 16: Analytics features supported by the Early Warning & Multi-Channel Alert Dissemination function.

6.3.3 Software Engineering

The Early Warning & Multi-Channel Alert Dissemination function adopts the same serverless, layered architecture used across FloodWatch, hosted on AWS, but its processing pipeline is event-driven and fan-out oriented rather than model-inference oriented. 2 independent event sources, an AWS IoT Core rule (direct threshold breach) and an Amazon EventBridge rule listening to the AI Prediction Engine's DynamoDB Stream (new high-risk report), both invoke the Warning Decision Lambda. This function-level decoupling means warnings keep going out even if the predictive pipeline is temporarily unavailable.

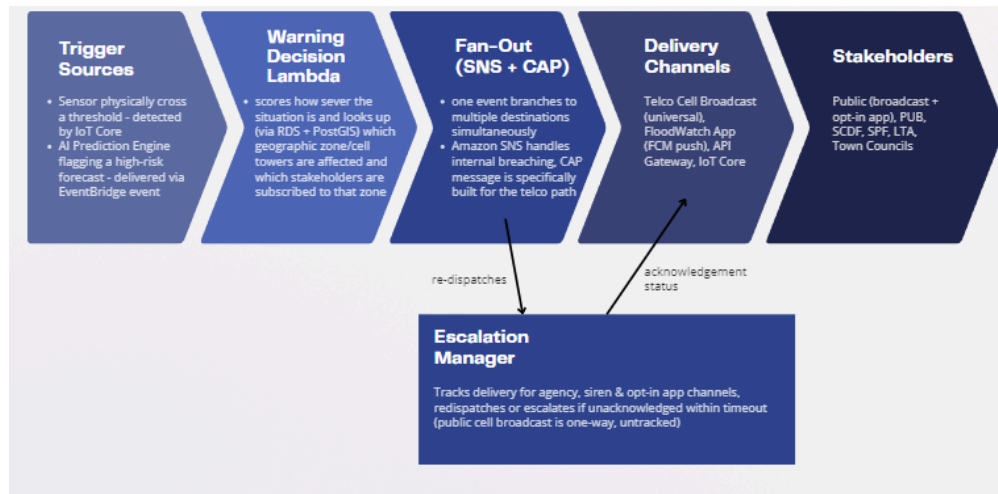


Figure 17. Architecture of the Early Warning & Multi-Channel Alert Dissemination Function

The Warning Decision Lambda scores the severity of the incoming event and determines 3 things in parallel:

1. which cell towers cover the affected flood zone (via the PostGIS Geofence Mapping),
2. which app users are subscribed to that zone (via RDS PostgreSQL), and
3. Which agencies are subscribed to that zone and severity tier (also via RDS PostgreSQL)

For the guaranteed public channel, it constructs a Common Alerting Protocol (CAP) message and sends it to the relevant telco's Cell Broadcast Entity, which broadcasts it directly to every compatible phone within the targeted cell towers, no app registration or phone number required, mirroring the carrier-operated cell broadcast approach used in Japan's Earthquake Early Warning System (GSMA 2026) and the United States' Wireless Emergency Alerts (Bean, Takenouchi & Cruz 2025).

In parallel, Amazon SNS fans the same event out to multiple delivery channels:

1. Firebase Cloud Messaging (FCM): Delivers richer content alerts to FloodWatch app users, including flood maps, alternative routes, and an in-app acknowledgement option.
2. Amazon API Gateway: Provides structured REST delivery to PUB, SCDF, SPF, LTA & Town Council Systems.
3. SMS Gateway: Sends alerts to field teams and on-call staff.
4. AWS IoT Core: For confirmed high-severity events, sends MQTT commands to activate physical sirens or electronic road signage in the affected zone.

Every dispatch, including the CAP message sent for public broadcast, is logged to DynamoDB and mirrored to Amazon QLDB for tamper-evident audit. As cell broadcast is inherently one-way, there is no per-device acknowledgement from that channel, instead the AWS Step Functions state machine, also known as the Escalation Manager, tracks outstanding acknowledgements for agency, siren/signage, and opt-in app channels, where a callback is

possible. If a High-severity alert is not acknowledged within its configured window, for example by SCDF, the state machine automatically re-publishes the alert to a secondary channel or escalates it to a higher-priority contact, closing the detect-to-knowledge loop for agencies and engaged app users without requiring manual follow-up.

6.3.4 Networking

6.3.4.1 Networking Infrastructure and Integration

The table below summarises the communication paths used by this function, each selected for the latency, reliability and security characteristics its purpose requires.

Source	Destination	Protocol	Rule/Purpose
AWS IoT Core (Threshold Rule)	Warning Decision Lambda	IoT Rule (Internal)	Forwards sensor readings that breach a configured threshold
AI Prediction Engine DynamoDB Stream	Warning Decision Lambda	Amazon EventBridge	Routes new high-risk flood prediction events to the alert engine
Warning Decision Lambda	Amazon SNS Topics	HTTPS (Internal AWS)	Publishes severity-scored alert for multi-channel fan-out
Warning Decision Lambda	Telco Cell Broadcast Entity (Singtel, Starhub, M1, SIMBA)	CAP (Common Alerting Protocol) over HTTPS	Broadcasts the warning to every compatible phone within cell towers covering the affected flood zone (no app registration or phone number required)
Amazon SNS	Firebase Cloud Messaging	HTTPS + JSON (FCM API)	Delivers richer push notifications (maps, routes) to opt-in FloodWatch app users
Amazon SNS	Agency Field-Team SMS Gateway	HTTPS/SMPP	Delivers SMS alerts to SCDF/LTA/Town Council field teams and on-call staff
Amazon SNS/API	Agency REST	HTTPS + API Key/	Delivers structured

Gateway	Endpoints (PUB, SCDF, SPF, LTA, Town Council)	IAM Auth	alerts to downstream agency systems
Warning Decision Lambda	AWS IoT Core (actuator command)	MQTT + TLS	Activates physical sirens/electronic signage in affected zones
Public App	API Gateway	HTTPS (TLS)	Opt-in subscription management and submission of delivery acknowledgements

Figure 18. Networking paths for the Early Warning & Multi-Channel Alert Dissemination function

Internal AWS-to-AWS hops (IoT Core to Lambda, EventBridge, Lambda to SNS) use managed, encrypted internal event routing rather than public internet paths, minimizing latency between detection and dispatch. The main internal hops are:

- AWS IoT Core → Lambda
- Lambda → Eventbridge
- Lambda → Amazon SNS

For external-facing channels, each delivery method uses a protocol suited to their endpoints:

1. Cell Broadcast: The guaranteed public warning is sent as a Common Alerting Protocol (CAP) message over HTTPS to the relevant telco's Cell Broadcast Entity. CAP is an international standard format also used by Japan's and the United States' cell-broadcast warning systems (Bean, Takenouchi & Cruz 2025)
2. FloodWatch App: The optional app channel uses the Firebase Cloud Messaging HTTPS API to deliver richer push content.
3. Agency systems: Alerts are delivered over HTTPS REST through Amazon API Gateway, secured using API keys or IAM-based authentication. This is consistent with how these agencies already integrate with the AI Prediction Engine's reports.
4. Field teams: SMS delivery relies on the gateway provider's HTTPS/SMPP interface.
5. Physical actuators: Sirens and electronic road signage receive MQTT commands over TLS through AWS IoT Core, mirroring the protocol already used for inbound sensor data.

6.3.4.2 Advanced Technologies and Security

Because a false or spoofed command could trigger unnecessary panic or, conversely, a suppressed alert could delay a genuine warning, this function treats command integrity and delivery accountability as first-class security requirements rather than an afterthought.

- Transport Layer Security (TLS) encrypts every hop, from sensor-to-cloud MQTT traffic to REST calls to agency systems and the CAP feed to telco Cell Broadcast Entities, preventing interception or tampering with warning content in transit.
- API keys and IAM-scoped permissions restrict which systems may call the agency-facing REST endpoints and which may invoke the actuator command topic, so only authorised FloodWatch components can trigger a siren or push an official warning.
- CAP messages sent to telco Cell Broadcast Entities are digitally signed and accepted only from FloodWatch's authenticated endpoint, preventing a spoofed or unauthorised flood warning from being broadcast to the public, a safeguard consistent with how cell-broadcast-based systems operate internationally.
- Message signing on actuator commands (siren and signage activation) allows AWS IoT Core to verify that a command genuinely originated from the Warning Decision Lambda before it is executed, preventing spoofed activation of physical infrastructure.
- The optional app channel uses token-based authentication so only a legitimately installed, registered app instance can receive push notifications or submit an acknowledgement on a resident's behalf.
- Dead-letter queues (via Amazon SQS) capture any agency or field-team alert that fails delivery on every attempted channel, ensuring it is never silently dropped and can be retried or escalated manually. Because public cell broadcast is inherently one-way, it cannot be tracked the same way; this is an accepted trade-off for reaching everyone in the area instantly, mirroring how existing cell-broadcast systems operate.
- An immutable audit ledger (Amazon QLDB) gives PUB and partner agencies a tamper-evident, cryptographically verifiable record of when each warning was dispatched, including public CAP broadcasts and agency acknowledgements. This is the function's equivalent of the AI engine's digital twin, where instead of a real-time visual model of physical infrastructure, it provides a real-time, trustworthy record of the warning process itself, which is important for post-incident accountability given that multiple independent agencies (PUB, SCDF, SPF, LTA, Town Councils) rely on the same warning stream to coordinate their response.

6.4 Stormwater Storage Optimisation

The Stormwater Storage Optimisation function is an AI-driven decision-support module within FloodWatch coordinates available stormwater storage before, during and after heavy rainfall. Instead of expanding physical canals, it optimizes existing detention tanks, retention ponds, reservoirs and public dual-use assets to maximise catchment capacity within PUB’s Source-Pathway-Receptor framework (PUB 2024). This approach draws on Singapore’s Stamford Detention Tank, which protects Orchard Road by storing severe runoff underground (CPG Consultants 2018).

The function combines rainfall forecasts from the AI-Based Flood Prediction & Risk Analysis function with real-time IoT sensor data, CCTV feeds and infrastructure status. AI models analyse these data to determine appropriate storage strategies throughout each stage of a rainfall event. A multimodal database stores spatial, graph, time-series and document data, while cloud computing enables scalable processing. A digital twin can simulate alternative strategies before deployment.

6.4.1 Operational Workflow

The function operates across three phases to maximize existing drainage capacity:

- 1. **Pre-rainfall Preparation:** Analyses forecasts, predicted runoff & storage capacity to create space before rainfall
- 2. **Real-time Stormwater Optimisation:** Monitor water levels, flow rates & storage utilisation nonstop to dynamically adjust storage strategies. CCTV analysis verifies that temporary storage areas are safe before activation.
- 3. **Post-rainfall Recovery:** Coordinates staggered dewatering sequence based on downstream capacity & asset priority to avoid secondary flooding

6.4.2 Multimodal Database

6.4.2.1 Dataset Storage

S/N	Data Type	Example Data	Database	Data Source	Reason
1	Spatial	Storage site boundaries, surrounding land use, elevation	PostgreSQL + PostGIS	Government GIS / PostGIS Layouts	Supports efficient spatial queries and GIS analysis
2	Graph	Drainage network topology (canals, pumps, reservoirs & water-flow connections)	Amazon Neptune	PUB Hydraulic Blueprint Network	Optimises connectivity and path traversal
3	Time-Series	Water levels, flow rates, rainfall intensity	Amazon Timestream	Holding Site Sensors & Gate Actuators	Handles high-frequency sensor data efficiently
4	Document	Storage asset capacity, maintenance logs, optimisation reports	Amazon DynamoDB	Land Use Datasets (SLA) & CCTV YOLOv11 Feeds	Flexible schema for semi-structured operational data

These database models are selected according to the characteristics of each dataset. PostGIS supports efficient spatial operations, Neptune enables network traversal, Timestream is optimised for high-frequency time-series data and DynamoDB provides scalable low-latency access to semi-structured operational data.

6.4.2.2 Dataset Query Processing

The function combines queries across these databases to support storage optimisation before, during and after rainfall.

S/N	Query Type	Database	Query Purpose	Example Query
1	Spatial	PostgreSQL + PostGIS	Identify available storage sites within a specified distance of a predicted flood location	Find available storage sites within 2 km of a predicted flood location.
2	Graph	Amazon Neptune	Determine feasible water-flow paths from an overloaded canal to available storage	Determine connected routes from an overloaded canal to available storage.
3	Time-Series	Amazon Timestream	Retrieve the latest water level, flow rate and storage utilisation for affected assets	Retrieve recent water-level and flow-rate trends.
4	Document	Amazon DynamoDB	Retrieve storage capacity, availability and maintenance status of candidate assets	Find available assets that are not under maintenance.

Spatial Query – Storage Site Selection

During **Phase 1**, PostgreSQL + PostGIS is queried to identify suitable temporary storage locations near a predicted flood area. Spatial functions such as `ST_DWithin()` can determine whether an available site is within a specified distance.

```
SELECT site_id, available_capacity_m3
FROM storage_sites
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_Point(103.XXXX, 1.XXXX), 4326),
    2000
)
AND available_capacity_m3 > 0;
```

The result is used by the optimisation engine to identify candidate storage sites.

Graph Query – Drainage Network Capacity

During **Phase 1**, Amazon Neptune is queried to trace the drainage network between a storage asset and downstream discharge points. Graph traversal allows the system to determine whether connected pathways have sufficient capacity before recommending water release or diversion.

```

g.V('SITE_JALAN_BESAR_STADIUM')
.outE('CONNECTS_TO')
.has('max_flow_capacity_m3_s', gt(15))
.inV()
.path()
.by('node_name')

```

This prevents the optimisation engine from selecting a storage strategy that could overload downstream drainage infrastructure.

Time-Series Query – Storage Monitoring

During **Phase 2**, Amazon Timestream continuously retrieves sensor readings from active storage sites. The system can calculate the rate at which water levels are increasing and determine whether a storage site is approaching its maximum capacity.

```

SELECT
  site_id,
  (MAX(measure_value::double) -
   MIN(measure_value::double)) / 10.0
  AS depth_increase_per_minute
FROM "FloodWatchDB"."StorageTelemetry"
WHERE measure_name = 'holding_water_depth_meters'
AND site_id = 'SITE_JALAN_BESAR_STADIUM'
AND time > NOW() - INTERVAL '10' MINUTE
GROUP BY site_id;

```

If the water level is rising too quickly, the optimisation engine can recalculate the storage allocation and adjust the diversion strategy.

Document Query – Asset Recovery

During **Phase 3**, Amazon DynamoDB is queried to identify storage assets that currently contain water and are ready for dewatering. Asset status, available capacity and maintenance information are used to determine the recovery sequence.

```

Filter:
waterPresent = true
AND dewateringPriorityIndex <= 2

```

The resulting assets are passed to the optimisation engine to determine a **staggered dewatering schedule**, preventing excessive water discharge into downstream drainage channels.

6.4.2.3 Analytics Supported

S/N	Analytics Type	Purpose	Main Data
1	Descriptive	Monitor current storage and drainage conditions	IoT time-series data
2	Predictive	Estimate future runoff and storage demand	AI-Based Flood Prediction Function, IoT Sensors

3	Spatial	Identify suitable storage locations	GIS Data, Spatial Data
4	Network(Graph)	Evaluate drainage connectivity and flow paths	Graph Data
5	Prescriptive	Recommend storage allocation and recovery strategies	AI Optimisation Engine

6.4.3 Software Design & Architecture

Figure 19 illustrates the software architecture of the Stormwater Storage Optimisation function.

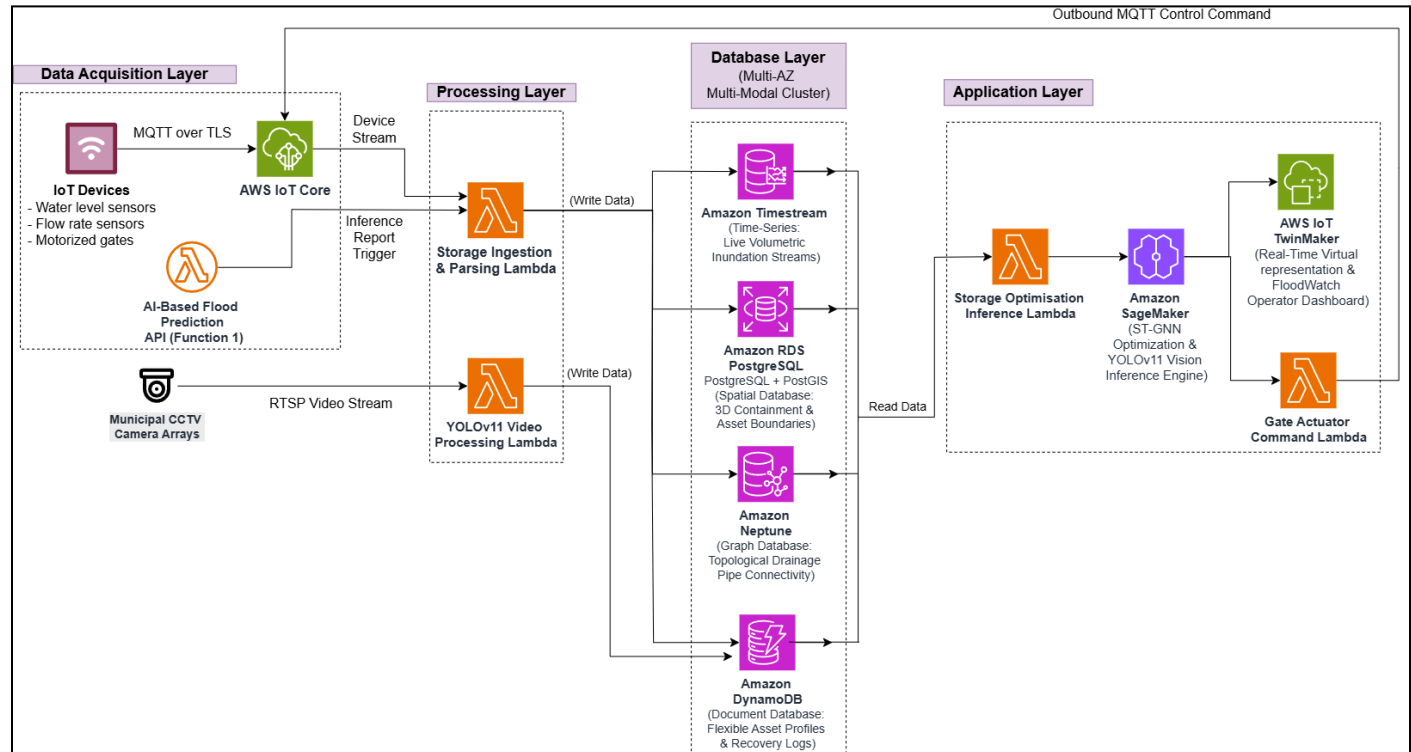


Figure 19. Software Architecture of the Stormwater Storage Optimisation Function

The Stormwater Storage Optimisation function uses a layered, event-driven serverless architecture deployed on AWS. It separates data acquisition, processing, storage and application services, while event-driven processing enables real-time responses to sensor and forecast updates. Serverless services provide automatic scaling during high-demand rainfall events.

6.4.3.1 Data Acquisition Layer

The Data Acquisition Layer is responsible for collecting environmental observations and operational information required by the optimisation engine. Two primary data sources are used:

- IoT water level, flow rate and storage utilisation sensors deployed throughout the drainage network.
- Rainfall forecasts and predicted runoff volumes generated by the AI-Based Flood Prediction & Risk Analysis function.

IoT water-level, flow-rate and storage sensors transmit telemetry to AWS IoT Core using MQTT over TLS. AWS IoT Core authenticates devices and forwards validated data to the cloud processing pipeline. Rainfall forecasts and predicted runoff are received through internal HTTPS APIs from the AI-Based Flood Prediction & Risk Analysis function.

6.4.3.2 Processing Layer

Sensor and forecast updates trigger AWS Lambda functions that validate, transform and route data into the multimodal databases. Significant changes in hydraulic conditions trigger the optimisation workflow. The processing layer retrieves the latest operational data from the multimodal databases and prepares the inputs required by the AI optimisation model.

6.4.3.3 Database Layer

The Database Layer stores operational data using a multimodal database architecture.

- Amazon Timestream stores real-time sensor observations.
- PostgreSQL with PostGIS stores spatial information describing storage assets & surrounding infrastructure.
- Amazon Neptune models the connectivity of Singapore's drainage network.
- Amazon DynamoDB stores operational metadata, optimisation recommendations & recovery logs.

6.4.3.4 Application Layer

The Application Layer contains the AI optimisation engine responsible for coordinating stormwater storage throughout the three operational phases.

The optimisation engine combines rainfall forecasts, sensor observations, network topology and spatial data to determine storage allocation strategies. The recommended operational plan is then transmitted to authorised PUB operators through the FloodWatch dashboard for monitoring and approval before control commands are issued to field infrastructure (see Figure 20).

When temporary storage assets require activation, the application layer invokes the computer vision service to analyse CCTV feeds and verify that designated storage areas are safe and unoccupied before recommendations are executed.

STORAGE CAPACITY OVERVIEW				
Asset	Current Level	Available	Recommended Action	Status
Detention Tank 01	62%	1,200 m³	Divert Inflow	Optimal
Canal 05	81%	340 m³	Monitor	Moderate
Storage Site 12	95%	40 m³	Activate Overflow	Critical

Figure 20. FloodWatch Operator Dashboard Asset Telemetry View, illustrating real-time PostGIS volume tracking, automated alert metrics for Canal 05 and algorithmic storage override recommendations.

6.4.3.5 Deployment Strategy

AWS Lambda provides serverless processing and orchestration, while Amazon SageMaker hosts the ST-GNN and YOLOv11 inference services. This enables independent model deployment and automatic scaling without dedicated servers.

6.4.3.6 Scalability & Fault Tolerance

Horizontal scalability is handled natively by the cloud architecture. AWS IoT Core securely scales to ingest simultaneous data streams from thousands of physical devices, while AWS Lambda dynamically provisions parallel execution environments to accommodate transactional volume spikes. Workloads are further optimized by the multi-modal database cluster, which distributes traffic across specialized data models instead of choking a single engine. To guarantee continuous fault tolerance during severe weather anomalies, all services are deployed across a Multi-Availability Zone (Multi-AZ) cluster with automated failover logic. Network interruptions are mitigated by AWS IoT Core, which buffers inbound edge metrics locally, while serverless functions execute automated message retry routines to eliminate data loss.

6.4.3.7 Application of AI for Software Engineering

The Stormwater Storage Optimisation function applies artificial intelligence to optimise the utilisation of stormwater storage assets before, during and after heavy rainfall events. Rather than predicting flood occurrence, the AI continuously analyses the state of the drainage network and recommends how existing storage resources should be coordinated to maximise available capacity while minimising downstream flood risk.

The optimisation process is deployed using **Amazon SageMaker**, which provides the infrastructure required to train, deploy & manage AI models within the AWS cloud environment. Real-time sensor observations, rainfall forecasts, spatial data and drainage network information are combined into a unified dataset before being processed by the optimisation models.

Two AI models are used within the optimisation workflow.

S/N	AI Model	Purpose
1	Spatio-Temporal Graph Neural Network (ST-GNN)	Optimises stormwater storage allocation and water diversion across the drainage network.
2	YOLOv11	Verifies that designated temporary storage areas are clear before activation.

Spatio-Temporal Graph Neural Network (ST-GNN)

The ST-GNN forms the core optimisation engine of the Stormwater Storage Optimisation function. Unlike conventional machine learning models that analyse individual sensor readings independently, ST-GNN represents Singapore's drainage infrastructure as an interconnected graph where hydraulic assets such as canals, detention tanks, reservoirs and temporary storage sites are represented as nodes, while water flow paths are represented as edges.

The model combines real-time sensor observations with historical and forecast data to learn both the spatial relationships between connected assets and the temporal evolution of hydraulic conditions. This enables the optimisation engine to determine how available storage capacity should be distributed throughout the network before, during and after heavy rainfall.

Input		Output
● Rainfall forecast ● Water Level ● Flow Rate	● Storage utilisation ● Drainage network topology	● Recommended Storage allocation ● Water diversion strategy ● Recovery sequence ● Storage cavity utilisation

YOLOv11

YOLOv11 performs real-time object detection on CCTV feeds to verify that temporary storage areas, such as a car park or open field, are clear before activation. Live CCTV images are analysed to detect pedestrians, vehicles and other obstacles within the designated storage area, preventing unsafe activation and reducing reliance on manual inspection.

6.4.4 Network Infrastructure and Integration

The Stormwater Storage Optimisation function relies on a reliable networking infrastructure to enable continuous communication between IoT devices, cloud services and operational users. The networking layer supports the collection of real-time environmental data, transmission of optimisation recommendations and coordination between multiple FloodWatch functions. To satisfy the system's low-latency and high-availability requirements, different communication protocols and network technologies are selected based on the characteristics of each data stream.

Communication Flow Mapping

Source	Destination	Communication Protocol	Purpose
IoT Sensors	AWS IoT Core	MQTT over TLS	Real-time sensor telemetry
CCTV Cameras	Edge AI / AWS	HTTPS (RTSP for video streaming)	Computer vision safety verification
AI-Based Flood Prediction Function	Optimisation Lambda	HTTPS REST API	Rainfall forecasts
AWS IoT Core	AWS Lambda	AWS IoT Rule	Event-driven processing
Optimisation Engine	PUB Dashboard	HTTPS	Display optimisation recommendations
Optimisation Engine	Gate/Pump Controllers	MQTT over TLS	Secure actuator commands

The networking layer uses various communication methods based on the type of data being transmitted. Water-level, flow-rate and storage sensors send small, frequent telemetry messages to AWS IoT Core using MQTT over TLS. MQTT is suitable for IoT devices as it has low

communication overhead, while TLS protects the transmitted data. AWS IoT Core supports MQTT with TLS and can use IoT Rules to route incoming messages to services such as Amazon Timestream and AWS Lambda (Amazon Web Services 2026a). CCTV feeds are processed at the edge using YOLOv11, reducing the need to send high-resolution video nonstop to the cloud and therefore lowering bandwidth usage and latency. Rainfall forecasts and predicted runoff from the AI-Based Flood Prediction & Risk Analysis function are transmitted to the optimisation function through REST APIs over HTTPS, providing secure communication between cloud services. Finally, AWS IoT Rules automatically trigger Lambda functions when relevant sensor conditions are detected, enabling event-driven processing without continuous polling.

Network Technologies

Network Technology	Purpose
5G	High-bandwidth, low-latency transmission for CCTV streams & actuator communication.
NB-IoT / LTE-M	Low-power communication for battery-powered environmental sensors.
Fibre / Ethernet	High-speed backbone connecting PUB infrastructure and cloud gateways.

The networking layer combines multiple communication technologies to satisfy different operational requirements. Low-power IoT sensors utilise NB-IoT (or LTE-M) to maximise battery life while maintaining wide-area coverage. CCTV cameras and operational control systems rely on 5G or fibre connectivity to support higher bandwidth requirements and minimise communication latency during flood events.

Requirement	FloodWatch Technical Implementation
Bandwidth	MQTT minimises bandwidth for sensors, while 5G supports high-definition CCTV streams.
Latency	Event-driven messaging & edge processing reduce response time during optimisation.
Reliability	MQTT QoS, TLS encryption & AWS IoT Core ensure reliable message delivery.
Coverage	NB-IoT and 5G provide connectivity across distributed drainage infrastructure.
Energy Efficiency	NB-IoT enables long battery life for remote environmental sensors.
Scalability	Publish-subscribe messaging & AWS IoT Core support thousands of connected devices.

6.4.5 Advanced Technologies and Security

Digital Twin

The Stormwater Storage Optimisation function incorporates digital twin technology to provide a real-time virtual representation of Singapore's stormwater storage network. AWS IoT TwinMaker

supports digital-twin entities that can integrate time-series, geospatial, video and other data sources to represent physical assets (Amazon Web Services 2026b).

The digital twin enables PUB operators to visualise water levels, storage utilisation and network connectivity in real time while simulating the impact of different storage allocation strategies before operational decisions are implemented. During heavy rainfall, optimisation recommendations generated by the AI engine are first reflected within the digital twin, allowing operators to evaluate the predicted effects on downstream drainage capacity and surrounding infrastructure before approving water diversion actions.

By integrating real-time monitoring with predictive simulation, the digital twin improves operational awareness and supports safer, more informed decision-making without disrupting the physical drainage network directly streaming high-priority alerts across connected nodes to field team widgets (see Figure 21).

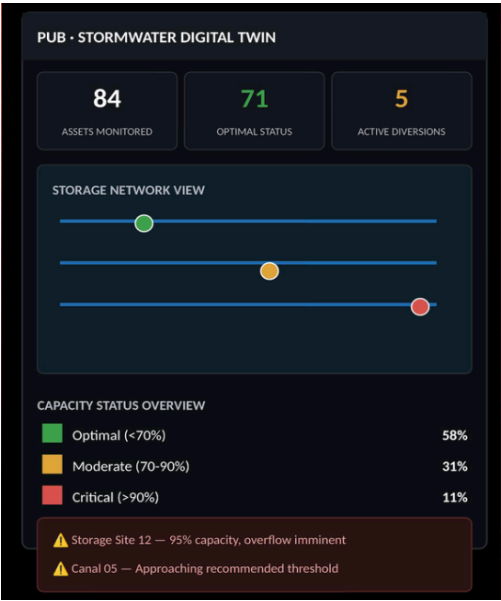


Figure 21. FloodWatch Mobile Command Widget via AWS IoT TwinMaker, illustrating synchronized time-series storage metrics, real-time node alerts for Storage Site 12 & active volumetric diversion loops.

Cybersecurity

Security Mechanism	Application
MQTT + TLS / X.509	Authenticates IoT devices and encrypts telemetry
HTTPS + API authentication	Secures inter-service communication
Sensor validation	Detects anomalous or inconsistent readings
IAM + RBAC + audit logs	Restricts access and records operational actions
Human-in-the-loop	Requires operator approval for critical physical actions

Together, these controls protect FloodWatch against unauthorised devices, data interception, false sensor readings and inappropriate operational commands.

6.5 Tactical Ground Situational Awareness & Operational Feeds

Frontline emergency teams receive dynamic coordination feeds and real-time geospatial data from the function. Although Function 1 predicts flood risk and Function 3 manages automated notification dispatch, this function converts live conditions into operational data that emergency responders can use :

- **Public Utilities Board (PUB) Quick Response Teams (QRT):** Locate active localized ponding locations and optimize the route access to install flood barriers.
- **Singapore Civil Defence Force (SCDF):** Navigate routes that avoid roads affected by flood during search, rescue and evacuation operations.
- **Land Transport Authority (LTA) & Traffic Police (SPF/Certis CISCO):** Locate affected road areas to set up road closures, block off flooded lanes and redirect traffic away from affected junctions.

6.5.1 Multi-Modal Database

6.5.1.1 Dataset Storage

S/N	Data Type	Data Schema	Database	Justification
1	Spatial Data (Geospatial)	Road and flood coordinates	Amazon RDS PostgreSQL + PostGIS	The functions are useful to identify which roads are near or affected by the flood.
2	Graph Data (Network Topology)	Road junction nodes and water path edges	Amazon Neptune	Less costly when calculating shortest routes not affected by flood and diversion of traffic.
3	Time-Series Data (Read-Only)	Water level, rain intensity and drainage readings	Amazon Timestream	Data is only added and never edited or deleted, so it can write data in a short time.
4	Document Data (Incident Reports)	Crowdsourced geo-tagged photos, reports made by citizens and field logs	Amazon DynamoDB	Supports JSON schemas that can store variable types (e.g. photo URLs, comments, status flags).

6.5.1.2 Dataset Query Processing

Spatial Query (Road Cordon)

SQL spatial functions like ST_Intersects and ST_Buffer are used to query geospatial data saved in PostgreSQL with the PostGIS extension. The system can quickly assess intersections between expanding flood areas and road paths without scanning the entire dataset thanks to spatial indexes (GiST) built on the road network geometries. This makes it possible to identify flooded

roadways in real time, which enables the system to quickly divert traffic by placing cordon boundaries to the Land Transport Authority (LTA) and Traffic Police.

Example Query : Retrieve all road segments affected by flood within Geylang.

Graph Query (Tactical Emergency Routing)

The OpenCypher query language in Amazon Neptune is used to query the connectivity of Singapore's road and transit network. Road intersections are represented as nodes, while road segments with associated transit times and traversal costs represent edges. The traversal cost of a road segment is adjusted when computer vision inference flags it as blocked or flooded. Graph traversal algorithms calculate passable, congestion-free paths for PUB Quick Response Teams and Singapore Civil Defense Force (SCDF) rescue trucks by executing shortest-path queries that automatically omit flooded nodes (Behrooz & Ilbeigi 2025).

Example Query : Execute the fastest passable route from Paya Lebar Fire Station to Geylang that avoids floods.

Time-Series Query (Read-Only)

To provide real-time metric overlays for tactical decision-making, real-time environmental telemetry stored in Amazon Timestream (filled by Function 1) is queried using SQL syntax. In order to compile recent water level elevations, rainfall rates and flow velocities over particular sectors, this function runs read-only queries. The extracted information provides frontline commanders with continuous trends without requiring this function to manage its own raw sensor ingestion pipelines.

Example Query : Fetch the data for a continuous 5-minute average water level and drainage velocity metrics for sensors within 1km of Geylang.

Document Query (Incident Reports & Tactical Field Logs)

DynamoDB document query APIs with composite partition and sort keys are used to query frontline responder incident reports, crowdsourced public flood submissions and semi-structured tactical logs stored in Amazon DynamoDB. The system can instantly retrieve unverified reports or search for active incidents by sector and timestamp due to attribute filters and Global Secondary Indexes (GSIs). The resulting JSON payloads provide the command dashboard with operational field context, such as submitted photo URLs, reporter comments and confirmation statuses.

Example Query : Retrieve all pending, unverified incident reports submitted at the Geylang sector within the last 5 minutes.

6.5.2 Software Design & Architecture

As seen from the figure below, the Tactical Ground Situational Awareness & Operational Feeds function uses the serverless, event-driven microservices architecture hosted on AWS Cloud.

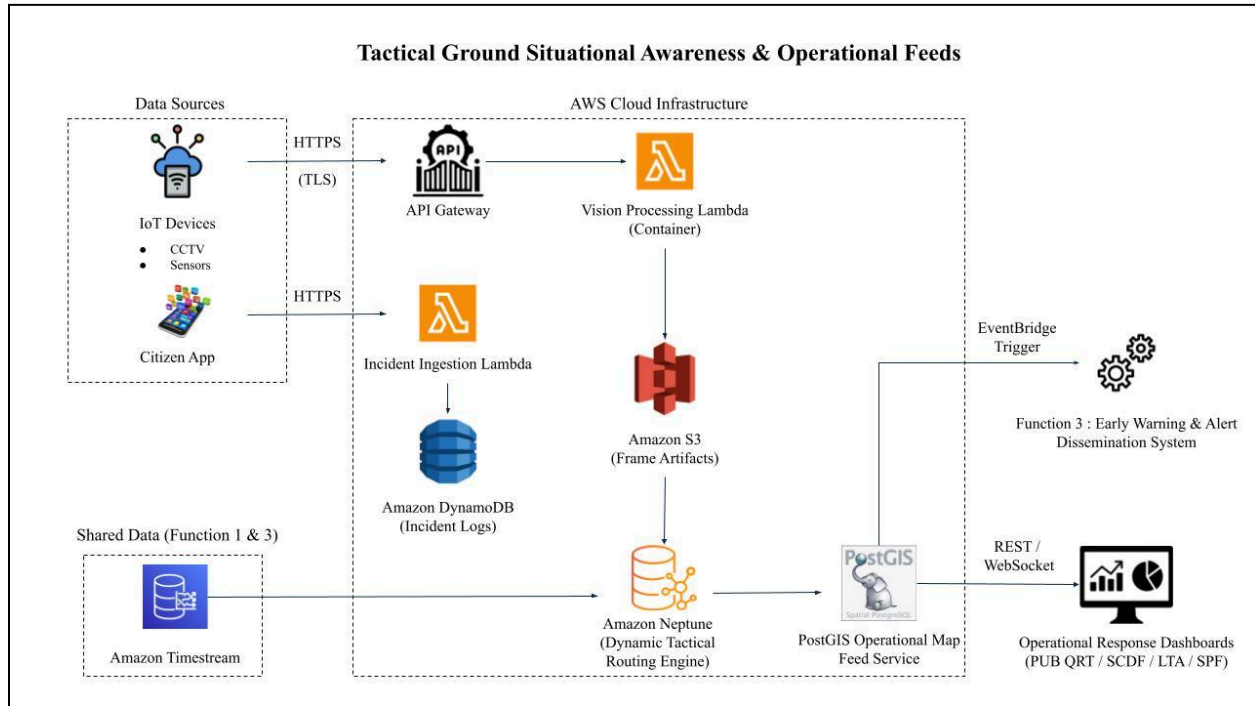


Figure 22. shows the architecture of the function

Interaction flow:

1. CCTV video frames from flood-affected roads are streamed over 5G/HTTPS to AWS API Gateway.
2. A computer vision processing Lambda function using an optimized YOLOv11 detector examines frames for obstructed cars, blocked roads, and flooding.
3. Road status updates are published to Amazon Neptune to update edge traversal weights, while geospatial boundaries are written to PostgreSQL + PostGIS.
4. Real-time operational map layers are served via WebSockets to command center dashboards used by PUB, SCDF, and LTA.
5. If severe road submersion is confirmed, an event is emitted via Amazon EventBridge to **Function 3**, triggering automated public cell broadcasts and agency-wide escalation protocols

6.5.2.1 Scalability & Fault Tolerance

During flash flood scenarios, server-less Lambda automatically scales concurrency based on user traffic and incoming visual telemetry. To maintain almost perfect operational uptime, relational and graph databases are spread over several AWS Availability Zones (Multi-AZ) using read-replicas.

6.5.2.2 Application of AI for Software Engineering

To automate ground condition verification and dynamic tactical routing, the Tactical Ground Situational Awareness & Operational Feeds function uses an optimized YOLOv11 object detection and computer vision model hosted on containerized Amazon SageMaker inference endpoints. The AI model continuously analyzes live, high-definition traffic CCTV frames broadcast from flood-prone roads and low-lying areas around Singapore, eliminating the need for manual operator inspections. The vision model performs bounding and classification to detect surface water accumulation, measure the extent of lane flooding, and identify obstructed vehicles (Yakin et al. 2025).

The model automatically updates the dynamic graph traversal weights in Amazon Neptune by marking the affected road edge as impassable when it detects severe lane flooding or a vehicle blockage with high inference certainty. The routing engine can instantly recalculate safe transit paths for emergency responders thanks to its real-time graph re-weighting, eliminating the need for human verification delays. Additionally, before the incident is ingested into Amazon DynamoDB, the computer vision pipeline automatically filters out out-of-context or fake imagery from crowdsourced incident photos uploaded via the citizen mobile app.

6.5.3 Networking Infrastructure & Integration

The Tactical Ground Situational Awareness & Operational Feeds function relies on a dependable networking layer that connects field-deployed cameras, incident-reporting applications, cloud analytical services, and frontline operational dashboards. Using HTTPS and RTSP over Transport Layer Security (TLS), high-definition traffic CCTV camera feeds placed at major roadways transmit video data via dedicated fiber backbones and high-bandwidth 5G connections. This prevents bandwidth bottlenecks while fulfilling the sub-second latency and high throughput requirements required for real-time computer vision inference on AWS.

Using REST APIs protected with TLS 1.3, inbound crowdsourced event reports and frontline responder updates from mobile devices are safely sent to AWS API Gateway over conventional 4G/5G mobile networks. The system leverages persistent WebSocket (WSS) connections to feed real-time PostGIS tactical map coordinates and dynamic rerouting updates to vehicle-mounted mobile data terminals (MDTs) used by SCDF and PUB QRT units, ensuring that frontline response teams are always informed in real time.

Source	Destination	Protocol	Network Technology	Purpose
Traffic CCTV Cameras	AWS API Gateway / S3	HTTPS/ RTSP (TLS)	5G/Fiber	Streaming surveillance frames at high speeds for AI vision analysis

Field Mobile Apps	AWS API Gateway	REST/HTTPS	4G/5G Mobile	Collecting geo-tagged citizen reports and responder field logs
PostGIS / Neptune Feed	Emergency Units (SCDF/PUB QRT)	WebSockets (WSS)	5G Mobile	Real-time streaming of live tactical maps.

6.5.4 Advanced Technologies & Security

Digital Twin Integration

The AWS IoT TwinMaker instance of the platform receives input from tactical geospatial layers. A real-time 3D situational twin of Singapore's metropolitan road network is available to frontline dispatchers, displaying closed traffic junctions, real-time water levels, and the GPS locations of responding QRT and SCDF units.

Cybersecurity

End-to-End Encryption

To prevent unauthorized eavesdropping and tampering, Transport Layer Security (TLS 1.3) is used to secure all CCTV video feeds, mobile incident reports, and real-time map data streams while they are in transit. Sensitive spatial records are protected from unwanted exfiltration by employing AES-256 managed keys to encrypt all stored operational datasets, map layers, and tactical incident documents in PostGIS, Amazon Neptune, and Amazon DynamoDB while it is at rest.

Role-Based Access Control (RBAC)

API and dashboard endpoints implement strict least-privilege policies using AWS Identity and Access Management (IAM) integrated with OAuth 2.0 and OpenID Connect (OIDC). This prevents unlawful command execution by ensuring that only authenticated frontline staff from approved agencies such as SCDF, SPF, LTA, and PUB can access operational maps or submit verified incident reports.

API Protection & Perimeter Defense

AWS Web Application Firewall (WAF) protects incoming network traffic via AWS API Gateway, inspecting and filtering all incoming API calls in real time. The firewall enforces rate limiting to mitigate Distributed Denial of Service (DDoS) attacks, while custom inspection rules actively detect and prevent SQL injection, cross-site scripting (XSS), and malicious file uploads from hacked or untrusted mobile clients.

7. References

Amazon Web Services 2026a, *AWS IoT Core - Amazon Timestream*, Amazon Web Services, viewed 4 August 2026,

<<https://docs.aws.amazon.com/timestream/latest/developerguide/IOT-Core.html>>

Amazon Web Services 2026b, *What is AWS IoT TwinMaker?*, Amazon Web Services, viewed 4 August 2026,

<<https://docs.aws.amazon.com/iot-twinmaker/latest/guide/what-is-twinmaker.html>>

BEAN, H, TAKENOUCI, K & CRUZ, AM 2025, 'Mobile public warning in Japan and the United States: a sister cities collaboration', *Frontiers in Communication*, vol. 10, article 1518729, viewed 5 August 2026, <<https://doi.org/10.3389/fcomm.2025.1518729>>

BEHROOZ, ILBEIGI 2025, *Adaptive emergency evacuation routing: a graph-based approach*, *International Journal of Transportation Science and Technology*, viewed 3 August 2026,

<<https://www.sciencedirect.com/science/article/pii/S2046043024001412>>

CPG Consultants 2018, *Stamford Detention Tank*, CPG Consultants, viewed 2 August 2026,

<<https://cpgconsultants.com.sg/blog/projects/stamford-detention-tank-singapore>>

GSMA 2026, *Japan's Early Warning System: The Role of Mobile Network Operators*, GSMA, viewed 3 August 2026,

<https://www.gsma.com/solutions-and-impact/connectivity-for-good/mobile-for-development/gsma_resources/japans-early-warning-system-the-role-of-mobile-network-operators/>

LOMMEN, D, WANG, J, SU, H, ZHI, X, WANG, X, PARK, E, ZHANG, H, LEE, J & CHYI, WM 2025, 'Burst flooding in Singapore: an emerging urban flooding type revealed by high-temporal-resolution observations', *Environmental Research Letters*, vol. 20, no. 1, article 014012, viewed 5 August 2026, <<https://doi.org/10.1088/1748-9326/ad975c>>

Public Utilities Board (PUB) 2024, *On-site Stormwater Detention Tank Systems: Technical Guide*, revision 3, PUB, viewed 1 August 2026,

<<https://www.pub.gov.sg/-/media/PUB/PDF/detentionTank.pdf>>

PUB 2026, *Flood forecasting and monitoring*, Public Utilities Board, viewed 2 August 2026,

<<https://www.pub.gov.sg/Public/KeyInitiatives/Flood-Resilience/Flood-Forecasting-and-Monitoring>>

YAKIN, SANTO, FAHIM, A M RUDRO, KAMRUDDIN 2025, *WLD-InceptionV3: AI and Computer Vision-Based Urban Waterlogging Detection Using a Modified InceptionV3 Model*, *Association for Computing Machinery*, viewed 1 August 2026,

<<https://dl.acm.org/doi/full/10.1145/3723178.3723183>>